XAI POWERED FRAUD DETECTION

A PROJECT REPORT

Submitted by

ASHISH SERU (1BM21CS035)
TANISHA GOTADKE (1BM21CS229)
SNEHA SANTHOSH BHAT (1BM21CS213)
ARCHIT MEHROTRA (1BM21CS031)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

Under the Guidance of

Dr. LATHA N. R.

Professor, BMSCE

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
2024-2025

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

Certified that the project entitled "**XAI POWERED FRAUD DETECTION**" is a bonafide work carried out by Ashish Seru (1BM21CS035), Tanisha Gotadke (1BM21CS229), Sneha Santhosh Bhat (1BM21CS213), and Archit Mehrotra (1BM21CS031) in partial fulfillment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belagavi during the academic year 2024-25. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said degree.

Guide

Dr. Latha N.R.
Professor,
Dept of CSE,
B.M.S. College of Engineering

Head of Department

Dr. KAVITHA SOODA,
Professor and HOD,
Dept of CSE,
B.M.S. College of Engineering

Principal

Dr. Bheemsha Arya
B.M.S. College of Engineering

External Viva

Name of the Examiners	Signature with Date
1.	
2.	

Table of Contents

TITLE	PAGE NO.
ABSTRACT	**i**
Declaration by the student batch and guide	**ii**
Acknowledgements	**iii**
LIST OF TABLES	**iv**
LIST OF FIGURES	**v**

CHAPTER NO.	TITLE	PAGE NO.
1	**Introduction**	1
1.1	Overview	1
1.2	Motivation	1
1.3	Objective	2
1.4	Scope	2
1.5	Existing System	3
1.6	Proposed System	4
2	**Literature Survey**	6
3	**Software and Hardware Requirement Specification**	16
3.1	Functional Requirements	16
3.2	Non-functional Requirements	18
3.3	Hardware Requirements	19
3.4	Software Requirements	19
3.5	Cost Estimation	20
4	**Design**	21
4.1	High Level Design	21
4.2	System Architecture	23
4.3	Use-case Diagram	28
4.4	Class Diagram	30
4.5	Sequence Diagram	34
4.6	Methodology	38
5	**Implementation**	40
5.1	Overview of Technologies Used	40
5.2	Implementation details of modules	43
5.3	Difficulties encountered and Strategies used to tackle	47
6	**Testing and Experimental Analysis and Results**	50
6.1	Unit Testing	50
6.2	Integration Testing	51
6.3	System Testing	53
6.4	Evaluation Metric and Performance Analysis	56

6.5	Experimental Dataset	59
7	**Conclusion and Future Enhancements**	61
7.1	Conclusion	61
7.2	Future Enhancements	61
	REFERENCES	62
	APPENDIX A: Snapshots	65
	APPENDIX B: Details of list of publications related to this project	69
	APENDIX C: Details of patents	70
	APENDIX D: Details of funding	70
	APENDIX E: POs and PSOs Mapped	71
	APENDIX F: Plagiarism report	72

Abstract

As financial operations become steadily concentrated online, the need for **strong, clear fraud detection systems** has become vital. Methods based on **rules** and **simple black-box models** used in traditional machine learning often do not perform well at detecting **complex, evolving fraud patterns**. Worse, they do not provide the needed **interpretability**—that is, **explanations of how and why decisions or predictions were made**—that is a critical requirement in domains like **finance** that are heavily regulated. This project presents an **emerging-technology solution** to the problem of undetected fraud: an **Explainable AI (XAI)-driven fraud detection system**.

Our method starts with **all-encompassing data preprocessing**, which handles **missing values** with a **SimpleImputer**, **standardizes features** with a **StandardScaler**, and **encodes categorical labels** with a **LabelEncoder**. **Anomaly detection** occurs with an **Isolation Forest**, which identifies **unusual transactions**. These flagged transactions are then **classified using a Random Forest model**. To improve clarity, the system employs **SHAP (SHapley Additive exPlanations)**, which offers **lucid, feature-level justifications** for every prediction it makes. Enabling this on the system is a simple **Streamlit interface**. It lets users see **which transactions have been flagged**, along with the **reasoning behind that decision**. Our solution is **efficient**, **interpretable**, and **scalable** in real-world settings. It combines **accurate fraud detection** with **explainability**. In doing so, it aims to enhance **stakeholder trust**, help in **complying with financial regulations**, and ensure the **responsible use of AI** in financial services. Stakeholder trust is in increased supply when they are secure in the knowledge that **accurate and interpretable fraud detection systems** are functioning as intended.

DECLARATION

We, hereby declare that the Major Project Phase-II work entitled **"XAI POWERED FRAUD DETECTION"** is a bonafide work and has been carried out by us under the guidance of Dr. Latha N. R.**,** Professor, Department of Computer Science and Engineering, B.M.S. College of Engineering, Bengaluru**,** in partial fulfillment of the requirements of the degree of Bachelor of Engineering in Computer Science and Engineering of Visvesvaraya Technological University, Belagavi.
I further declare that, to the best of my knowledge and belief, this project has not been submitted either in part or in full to any other university for the award of any degree.

Candidate details:

SL. NO.	Student Name	USN	Student's Signature
1	Ashish Seru	1BM21CS035	
2	Tanisha Gotadke	1BM21CS229	
3	Sneha Santhosh Bhat	1BM21CS213	
4	Archit Mehrotra	1BM21CS031	

Place: Bengaluru

Date:23/05/2025

Certified that these candidates are students of Computer Science and Engineering Department of B.M.S. College of Engineering. They have carried out the project work of titled "XAI Powered Fraud Detection" as Major Project Phase-II work. It is in partial fulfillment for completing the requirement for the award of B.E. degree by VTU. The works is original and duly certify the same.

Guide Name Signature

Dr. Latha N. R.

Date:

Acknowledgment

We would like to express our sincere gratitude to all the individuals who supported us and provided valuable insights during the literature survey and throughout the development of this project. Their contributions have been instrumental in shaping our understanding and refining our approach to fraud detection using Explainable AI (XAI).

We are deeply grateful to our project guide, Dr. Latha N. R., Associate Professor in the Department of Computer Science and Engineering, for her constant support and guidance. Her encouragement and expertise played a crucial role in shaping our project. Dr. Latha's thoughtful feedback and steady presence helped us navigate challenges and stay on track. Her dedication and positivity have truly inspired us throughout this journey.

.

List of tables

Table No.	Description	Page No.
1	Literature Survey of XAI Powered Fraud Detection Systems	6
2	Cost Estimation of the proposed system	20

List of figures

Figure No.	Description	Page No.
1	High Level Design	21
2	Abstraction of the completed system	22
3	System Architecture	23
4	Use Case Diagram	28
5	Class Diagram	30
6	Sequence Diagram	34
7	Detailed Sequence Diagram	37
8	Output : Metrics for each fold	57
9	Output : Average metrics across fold	57
10	Output : Feature Importance Table	58
11	Sample Dataset	60

Chapter 1

Introduction

1.1 Overview

Financial fraud is a growing concern in digital transactions, leading to significant financial losses and regulatory challenges. Traditional fraud detection systems, often rule-based or statistical, struggle with high false positives, evolving fraud tactics, and a lack of transparency in decision-making. This project proposes a **Hybrid Explainable AI (XAI) Fraud Detection System** that integrates machine learning, anomaly detection, and interpretability techniques to enhance fraud detection accuracy while ensuring regulatory compliance. By leveraging **SHAP (SHapley Additive Explanations)** the system provides clear insights into fraud detection decisions, making AI-driven processes more trustworthy and transparent. The proposed solution is designed to improve fraud detection efficiency, minimize operational costs, and increase stakeholder confidence in financial institutions.

1.2 Motivation

In today's digital era, **financial transactions** have become an integral part of everyday life, from **online shopping** to **instant bank transfers**. However, with this growing convenience comes an alarming rise in **fraudulent activities**, costing businesses and individuals **billions of dollars** each year. **Traditional fraud detection systems**, which rely on **predefined rules** and **statistical models**, often fail to keep up with **evolving fraud tactics**. These methods generate a high number of **false positives**, leading to **unnecessary transaction declines**, **frustrated customers**, and **financial institutions** struggling to balance **security** with a **seamless user experience**.

One of the biggest challenges in **modern fraud detection** is the **lack of transparency** in **AI-driven models**. While **machine learning** has significantly improved **fraud detection accuracy**, most models operate as **"black boxes,"** making it difficult for **analysts** and **regulators** to understand why a particular transaction was flagged as **fraudulent**. This **lack of explainability** not only raises concerns about **bias and fairness** but also creates **compliance challenges**, as financial institutions must adhere to strict **regulatory frameworks** like **GDPR** and **RBI guidelines**.

This initiative is propelled by the urgency of creating an **interpretable fraud detection system** that not only identifies fraud with **precision** but also sheds light on the **reasons behind fraud-related conclusions**. By implementing **Explainable AI (XAI)** methods like **SHAP (SHapley Additive Explanations)**, our system will have **lucid explanations** regarding the **processes through which fraud-related decisions are taken**. This will assist financial institutions in keeping their **losses low**, **streamlining their costs**, and **fostering customer confidence**. Ultimately, our vision is to build a **stable fraud detection solution** that improves **security** while promoting **transparency**—making **AI-powered fraud prevention** more **effective and responsible**.

1.3 Objective

The objectives of this project are as follows:

- **Develop a Highly Accurate Fraud Detection System** – Minimize false positives and negatives using machine learning.
- **Implement Anomaly Detection** – Identify suspicious transaction patterns using unsupervised learning.
- **Enhance Interpretability with Explainable AI (XAI)** – Use **SHAP** to explain fraud detection decisions.
- **Minimize Financial Losses and Operational Costs** – Reduce fraud-related losses and improve detection efficiency.
- **Optimize System Performance for Large-Scale Transactions** – Ensure scalability for high transaction volumes.
- **Enhance Trust and User Confidence** – Provide transparent and explainable fraud detection decisions.

1.4 Scope

With the increasing reliance on digital transactions, financial fraud has become a major concern for individuals, businesses, and financial institutions. This project aims to develop a fraud detection system that effectively identifies fraudulent transactions in **online banking, credit**

card payments, and digital wallets. By leveraging machine learning and Explainable AI (XAI), the system will provide **highly accurate fraud detection while ensuring transparency and interpretability**.

The project will incorporate **anomaly detection techniques** to identify suspicious patterns and **supervised classification models** to distinguish between legitimate and fraudulent transactions. To address the lack of explainability in traditional AI models, the system will integrate **SHAP (SHapley Additive Explanations)** to provide clear insights into why a transaction is flagged as fraudulent.

Beyond fraud detection, the project is designed to **align with financial industry regulations** such as **GDPR, RBI guidelines, and Anti-Money Laundering (AML) laws**, ensuring compliance and building trust with users. The system will be scalable to handle **large volumes of transactions in real-time or near real-time**, making it suitable for banks, payment processors, and fintech companies. Ultimately, this project aims to create a **secure, efficient, and transparent fraud detection system** that enhances security while maintaining compliance with regulatory requirements.

1.5 Existing System

The current fraud detection systems used by financial institutions and online payment platforms primarily rely on **rule-based models and traditional machine learning techniques**. These systems work by defining **pre-set rules**, such as flagging transactions above a certain threshold or detecting multiple transactions from different locations in a short period. While these approaches have been effective to some extent, they suffer from several limitations.

One major drawback of rule-based systems is their **rigidity and lack of adaptability** to evolving fraud patterns. Fraudsters continuously develop new strategies to bypass detection, and static rules cannot keep up with these changes. Additionally, these systems generate **a high number of false positives**, causing legitimate transactions to be incorrectly flagged as fraudulent, leading to inconvenience for customers and financial losses for businesses.

Some financial institutions have adopted **basic machine learning models** for fraud detection,

but these models often operate as **black boxes**, meaning they make predictions without explaining why a transaction was classified as fraudulent. This lack of transparency makes it difficult for financial analysts and regulators to **trust and validate** the system's decisions, which is crucial for ensuring compliance with financial laws and regulations.

Overall, existing fraud detection systems struggle with **high false positive rates, poor adaptability to emerging fraud tactics, and a lack of explainability**, making them inefficient in detecting modern financial fraud in real-time.

1.6 Proposed System

To tackle the **limitations of traditional fraud detection systems**, we're building a **fraud detection solution powered by Explainable AI (XAI)**. The goal is to improve **detection accuracy** while keeping the system **transparent**, **adaptable**, and **compliant with financial regulations**. Unlike **rigid rule-based systems** or **black-box AI models**, our approach can **learn from new fraud patterns** and clearly **explain its decisions**.

We begin with **detailed data preprocessing and feature engineering**. This involves **cleaning and normalizing transaction data** and incorporating relevant **behavioral features** such as **frequency of transactions**, **location patterns**, and **device usage**. **Missing values** are handled using **SimpleImputer** from `sklearn.impute`, which imputes gaps based on **mean**, **median**, or **most frequent value** strategies. **StandardScaler** is applied to **normalize the features** to a uniform scale, which aids our models in performing better. We also apply **LabelEncoder** to transform **categorical class labels into integers**.

For detecting anomalies, we employ **Isolation Forest**, which is good at detecting **abnormal patterns** that can indicate fraud, particularly when fraudsters attempt to **emulate normal behavior**. After the **potential anomalies** are detected, we employ a **Random Forest classifier** to make the final determination of whether a transaction is **fraudulent or not**. **Random Forest** is a **robust**, **interpretable model** that works well with both **numerical** and **categorical data**.

What truly differentiates our system is the incorporation of **SHAP (SHapley Additive Explanations)** for **explainability** of the model. **SHAP** allows us to know which of the **features**

was most responsible for a particular prediction, enabling **analysts** and **regulatory agencies** to **trust and verify** the system's output more easily. We have also constructed a **clean and interactive Streamlit dashboard** through which **stakeholders** can **monitor the system**, **inspect predictions**, and see **SHAP-based explanations in real time**. The system is **scalable** and can be deployed on **cloud platforms** such as **AWS** or **Google Cloud**, or integrated directly into the **financial institution's infrastructure**.

In short, our solution is **precise**, **transparent**, and **ready for deployment** in **real-world environments**. By integrating **strong machine learning** with **explainability**, we seek to enable **financial institutions** to **detect fraud more effectively** while ensuring **trust** and **regulatory compliance.**

Chapter 2
Literature Survey

This below Table 2.1 provides the details about all the relevant papers to our topic, XAI Powered Fraud Detection.

S.No.	Paper Title	Year of Publication	Problem Statement Addressed	Methodology Followed	Quantitative Results	Future Work Proposed
1.	A suspicious financial transaction detection model using autoencoder and risk-based approach [1]	2024	Limitations of traditional AML systems in detecting anomalies in financial transactions, especially in imbalanced datasets.	Combines Autoencoder for anomaly detection with a Risk-Based Approach (RBA) for evaluating transaction risks based on reconstruction errors.	Outperformed traditional methods in accuracy and efficiency, especially for real-time data.	Explore multi-modal datasets and integrate explainable AI techniques for better transparency.
2.	Deep Neural Networks for Anti Money Laundering Using Explainable Artificial Intelligence [4]	2024	Challenges in applying Deep Neural Networks (DNNs) for detecting money laundering due to their lack of interpretability and transparency.	Integrates DNNs with SHAP to explain predictions while enhancing anomaly detection accuracy.	SHAP-enhanced DNNs achieve better detection performance and improved interpretability compared to rule-based methods.	Apply XAI techniques to other advanced models and test performance with real-world datasets.

3.	Enhancing Money Laundering Detection Addressing Imbalanced Data and Leveraging Typological Features Analysis [5]	2024	Difficulty in detecting fraudulent transactions in imbalanced datasets common in AML tasks.	Uses SMOTE to address class imbalance and improves recall rates by generating synthetic samples for minority class instances.	Significant improvement in recall for detecting minority class transactions; reduced false negatives.	Further research on reducing overfitting caused by oversampling and testing on diverse datasets.
4.	Improving Classificatio n Performance of Money Laundering Transactions Using Typological Features [6]	2023	Challenges in improving classification accuracy for suspicious financial transactions in AML contexts.	Compares Random Forest and XGBoost on resampled datasets, emphasizing the importance of typological features in classification.	Random Forest and XGBoost achieved high precision and recall, especially with typological feature integration.	Incorporate real-time transaction monitoring and adaptive learning for evolving fraud patterns.
5.	Towards responsible AI for financial transactions [7]	2020	Lack of explainability in financial AI systems, leading to trust and compliance issues.	Proposes integrating SHAP into fraud detection workflows to enhance transparency and	Improved stakeholder trust and compliance through SHAP explanations integrated with financial AI	Develop scalable explainability frameworks for complex, real-time financial AI applications.

				operational efficiency in AML systems.	systems.	
6.	Financial crime & fraud detection using graph computing: Application consideratio ns & outlook [13]	2020	Difficulty in detecting hidden patterns of laundering within complex financial transaction networks.	Employs Graph Neural Networks (GNNs) for detecting money laundering patterns, incorporating SHAP for improving interpretability .	High precision in identifying complex laundering patterns; scalability challenges noted.	Address real-time scalability issues and test GNN-based methods on diverse and large-scale datasets.
7.	Deep learning and explainable artificial intelligence techniques applied for detecting money laundering– a critical review [17]	2021	Trade-offs between model performance and interpretability in applying deep learning techniques for AML detection.	Reviews SHAP and LIME, highlighting SHAP's rigor and LIME's efficiency, and emphasizes the integration of explainable models with deep learning methods.	SHAP provides reliable explanations for deep learning models, while LIME is faster for local explanations.	Optimize SHAP for real-time applications and address computational overhead challenges in large datasets.

8.	Explainable AI-enhanced ensemble learning for financial fraud detection in mobile money transactions [20]	2024	Lack of transparency in fraud detection models used in mobile money transactions, affecting trust and compliance.	Combines ensemble learning models with SHAP to provide interpretability and achieve high accuracy in detecting fraudulent mobile money transactions.	Achieved 99.9% accuracy; SHAP explanations enhanced interpretability and feature-level insights.	Extend ensemble models with adaptive mechanisms to detect evolving fraud patterns.
9.	Opportunities in real time fraud detection: An explainable artificial intelligence (XAI) research agenda [24]	2023	Challenges in addressing real-time fraud detection, including concept drift, class imbalance, and scalability.	Proposes a research agenda emphasizing the integration of XAI techniques to address operational challenges and improve model transparency in real-time systems.	Outlines key priorities for XAI, focusing on scalability and explainability in real-time fraud detection systems.	Explore hybrid intrinsic explainable models that balance performance and interpretability.
10.	Explainable machine learning for	2021	Difficulty in balancing runtime	Compares SHAP and LIME in	SHAP offers superior reliability;	Investigate optimal dataset selection

	fraud detection [25]		efficiency and explanation reliability in fraud detection systems using XAI.	explaining fraud detection models, with SHAP providing more reliable global explanations and LIME generating faster local explanations.	LIME excels in speed but lacks robustness for complex models.	strategies for improving explanation quality and model adaptability.
11.	A Hybrid Machine Learning Approach for Real-Time Fraud Detection in Online Payment Transactions [26]	2024	Issues in detecting minority fraud cases in real-time online payment systems due to imbalanced datasets and evolving fraud patterns.	Combines Autoencoders for unsupervised feature extraction with Gradient Boosting for supervised classification, achieving high accuracy and low latency per transaction.	Achieved 99.5% accuracy; latency reduced to 2.8 ms per transaction.	Develop adaptive hybrid models to handle dynamic fraud patterns and improve interpretability of decisions.

Table 1. Literature Survey of XAI Powered Fraud Detection Systems

The following section briefly explains the various research works on XAI powered fraud detection, and machine learning techniques for fraud detection, focusing on financial systems. The literature survey highlights key methodologies, datasets, and results relevant to achieving

high accuracy and transparency in fraud detection while ensuring regulatory compliance and minimizing operational costs.

This paper [1] introduces an Autoencoder-based anomaly detection model integrated with a Risk-Based Approach (RBA) for Anti-Money Laundering (AML). The model autonomously learns transaction patterns and identifies anomalies by reconstructing input data and evaluating reconstruction errors. It addresses the limitations of rule-based systems, such as their rigidity and inability to adapt to evolving fraud tactics. The study also highlights the Autoencoder's superior performance in handling real-time transaction data, though challenges like high computational demands are noted.

This paper [2] analyzes the integration of AI-driven tools in the banking sector, focusing on enhancing operational efficiency, customer experience, and compliance in financial systems. It highlights applications such as SAS Fraud Management and Actimize for AML and discusses how AI tools, including blockchain and cybersecurity systems, improve fraud detection and transaction integrity.

This study [3] examines the application of advanced AI algorithms for detecting financial crimes like money laundering. It reviews hybrid AI methods combining supervised and unsupervised techniques to reduce false positives and enhance detection accuracy, particularly by analyzing transaction-specific and client-specific features.

This study [4] investigates the combination of Deep Neural Networks (DNNs) with SHAP (SHapley Additive exPlanations) to enhance anomaly detection in AML. DNNs are employed to detect anomalies in financial transactions, with SHAP providing insights into the decision-making process. The study addresses the "black-box" nature of DNNs by explaining key features influencing the detection of anomalies. This work emphasizes the importance of explainability for compliance with regulations and increasing trust in AI systems.

This paper [5] focuses on how Synthetic Minority Oversampling Technique (SMOTE) helps with handling imbalanced datasets in the context of money laundering detection. It demonstrates how SMOTE generates synthetic data to improve recall rates for minority class detection, such as

fraudulent transactions. The paper also discusses the risk of overfitting and the challenges in generalizing models across datasets, emphasizing the need for carefully calibrated oversampling strategies.

This paper [6] evaluates the effectiveness of Random Forest and XGBoost classifiers for fraud detection. Both models are tested on imbalanced datasets, and the study highlights their performance when used alongside SMOTE for resampling. Random Forest and XGBoost exhibit high precision and recall in identifying fraudulent transactions, showcasing their utility for classification tasks in financial crime detection.

This paper [7] emphasizes the role of responsible AI in fraud detection, proposing the integration of SHAP into fraud detection systems to improve transparency and operational compliance. It discusses how trust in AI models can be enhanced through explainability, addressing regulatory requirements.

This paper [8] introduces Variational Autoencoders (VAEs) and Wasserstein Generative Adversarial Networks (WGANs) for improving AML processes. The generative models address data imbalance by creating synthetic samples, improving detection accuracy for anomalies in imbalanced datasets.

This paper [9] investigates the use of Neural Networks (NN) for profiling money laundering activities associated with environmental crimes. The NN model identifies laundering patterns with high precision and employs incremental learning to adapt to dynamic real-world datasets, improving scalability and anomaly detection.

This study [10] focuses on predicting loan defaults using machine learning techniques such as logistic regression, decision trees, and ensemble methods. The research demonstrates that ensemble methods significantly outperform traditional statistical models, improving risk assessment accuracy while highlighting interpretability challenges in complex models.

This paper [11] compares traditional rule-based AML systems with machine learning approaches, including supervised and unsupervised models. It emphasizes that machine learning models like

Decision Trees and Neural Networks provide greater accuracy and adaptability, although scalability and computational overhead remain challenges.

This study [12] investigates the use of Generative AI and Large Language Models (LLMs) in AML systems, with a focus on balancing anomaly detection accuracy and privacy compliance. It leverages differential privacy frameworks and anomaly detection techniques to identify suspicious transactions while addressing data security and scalability concerns ￼.

This study [13] explores the application of Graph Neural Networks (GNNs) for detecting complex laundering patterns in transaction networks. By incorporating SHAP, the study enhances the interpretability of GNNs, which are typically seen as black-box models. While GNNs show promise for capturing intricate relationships in transaction data, scalability remains a significant challenge for real-time applications.

This paper [14] applies the Weighted HITS algorithm to detect suspicious customers involved in money laundering activities by analyzing transaction network centrality. The method accurately ranks high-risk customers based on transactional irregularities, but its scalability is limited for larger datasets .

This research [15] explores the use of Artificial Intelligence for AML and asset recovery. It integrates predictive models with anomaly detection to improve the efficiency of Suspicious Activity Reports (SARs) and reduce false positives.

This study [16] applies group-aware deep graph learning for AML, highlighting the importance of analyzing group interactions in transaction data. The proposed graph neural network significantly improves anomaly detection by leveraging group dynamics but notes challenges in generalizing across diverse datasets.

This paper [17] critically reviews the application of SHAP and LIME in fraud detection systems. SHAP is highlighted as a mathematically rigorous and reliable method for generating feature importance explanations, while LIME is noted for its faster runtime and simplicity. The study emphasizes the need for integrating XAI into AML systems to enhance transparency and

regulatory compliance, while addressing challenges like scalability and computational costs.

This paper [18] examines the application of Convolutional Neural Networks (CNNs) combined with SHAP to enhance interpretability in AML systems. The study highlights how SHAP can explain CNN predictions, improving operational efficiency and compliance in financial institutions.

This study [19] evaluates perspectives on XAI in the financial sector, with a focus on AML applications. It discusses challenges like regulatory uncertainty, the slow adoption of AI systems, and the need for a mindset shift from traditional to probabilistic thinking in fraud detection.

This paper [20] develops an ensemble learning model combining SHAP for feature interpretability and machine learning for fraud detection in mobile transactions. The model achieves an accuracy of 99.9% and bridges the gap between precision and transparency. It demonstrates how SHAP explanations can provide actionable insights into fraudulent activities, making it a valuable tool for decision-making in financial systems.

This study [21] investigates the potential of graph-based AI systems with attention mechanisms for detecting financial fraud. The use of multi-stage graph convolutional networks highlights the ability to capture and explain complex interactions in transactional networks. The paper emphasizes scalability and explainability as key challenges in adopting these systems in real-world scenarios.

This research [22] develops a fully convolutional network (FCN) for real-time fraud detection in continuous transaction streams. The FCN eliminates the need for pre-training by directly learning both spatial and temporal features. It incorporates gloss feature enhancement techniques to improve anomaly detection and reduce false positives.

This paper [23] applies Explainable Machine Learning Techniques (SHAP and LIME) to evaluate their effectiveness in fraud detection systems. The study highlights the trade-offs between accuracy and interpretability, emphasizing the importance of balancing these aspects in high-stakes domains.

This paper [24] outlines a research agenda focusing on the role of Explainable Artificial Intelligence (XAI) in addressing real-time fraud detection challenges such as concept drift, class imbalance, and scalability. It advocates for the development of intrinsic explainability in fraud detection models to improve their operational effectiveness and regulatory compliance.

This study [25] evaluates SHAP and LIME in real-time fraud detection using the IEEE-CIS Fraud Detection dataset. It finds that SHAP generates more reliable and detailed explanations than LIME, although LIME is faster for generating local explanations. The research highlights the importance of selecting appropriate background datasets for explanations, which significantly affect model performance and interpretability.

This study [26] proposes a hybrid model that combines Autoencoders for unsupervised feature extraction with Gradient Boosting for supervised classification. The model achieves an accuracy of 99.5% and demonstrates real-time processing capabilities with low latency (2.8 ms per transaction). However, the study acknowledges challenges in recall for minority fraud cases and notes the limited interpretability of the hybrid system's decisions, which impacts stakeholder trust .

Research Gaps and Challenges:

- Limited Adoption of Explainable AI: Papers [17] and [25] emphasize the computational overhead of SHAP and the lack of scalability in current XAI methods. Further research is needed to optimize XAI techniques for real-time applications.
- Balancing Accuracy with Interpretability: Papers [4], [13], and [26] note the challenge of maintaining high model performance while ensuring transparency, especially in high-dimensional datasets.
- Evolving Fraud Patterns: Papers [1] and [20] highlight the need for adaptive learning systems to handle the ever-changing nature of fraudulent activities, ensuring resilience and reliability in detection systems.

Chapter 3
Software and Hardware Requirement Specification

3.1 Functional Requirements

1. Data Preprocessing

FR-1.1: Missing Data Handling

- Missing values (NaN) must be replaced by configurable strategies, i.e., mean, median, most frequent, and constant value imputation.
- The preprocessing part shall utilize `SimpleImputer` from `sklearn.impute` to handle missing values.

FR-1.2: Feature Scaling

- Numerical features must be standardized by subtracting the mean and scaling to unit variance.
- `StandardScaler` from `sklearn.preprocessing` shall be employed to have uniform feature contribution while training.

FR-1.3: Encoding Labels

- The system needs to transform categorical class labels into numeric form through label encoding.
- `LabelEncoder` from `sklearn.preprocessing` will be used to encode categorical variables.

2. Anomaly Detection

FR-2.1: Isolation Forest Implementation

- The system needs to use Isolation Forest for anomaly and outlier detection representing fraudulent activity.
- The anomaly detection module will output anomaly scores for every instance, representing the level of isolation.

3. Classification Layer

FR-3.1: Random Forest Classifier

- The system will need to have a Random Forest classifier that classifies transactions/data points as "Fraud" or "Not Fraud."
- Classification outputs will be both binary decisions and corresponding fraud percentage scores.

4. Explainable AI (XAI)

FR-4.1: SHAP Integration for Interpretability

- The platform will need to incorporate SHAP (SHapley Additive exPlanations) to provide model interpretability and transparency.
- SHAP values will be calculated to determine feature importance driving individual and collective predictions.

FR-4.2: Interactive Visualization of Explanations

- The system will offer interactive visualizations (e.g., bar plots, beeswarm plots) of SHAP values to easily interpret feature effects.

5. User Interface (Streamlit Dashboard)

FR-5.1: Interactive Dashboard

- The system should provide an easy-to-use interactive dashboard developed with Streamlit where users can upload datasets, start preprocessing, anomaly detection, classification, and XAI visualization.

3.2 Non-functional Requirements

Performance:

- The system should process transactions efficiently but does not need real-time processing.
- Target response time: **Under 1 seconds per transaction**.

Scalability:

- It can be integrated with API's to perform real time analysis and provide accurate predictions.

Security & Privacy:

- No **real-time banking integration**, so security concerns are limited.

Reliability & Maintainability:

- The project should be able to run smoothly on a **college lab setup** or a personal laptop.
- Use of **GitHub for version control** to maintain project progress.

Compliance & Documentation:

- While not integrated with real banking systems, the project should **simulate compliance** with financial regulations (e.g., RBI guidelines on fraud detection).
- Proper documentation of **model decisions (SHAP outputs)** to ensure transparency.

3.3 Hardware Requirements

Component	Minimum Specification
Processor	Intel i5/i7 or Ryzen 5/7
RAM	8GB (Recommended: 16GB)
Storage	256GB SSD or higher
GPU (if used for ML training)	Not required (but optional for faster training: NVIDIA GTX 1650+)
Deployment	Local system (Jupyter Notebook / Google Colab)

3.4 Software Requirements

Software	Version
Programming Language	Python 3.8+
ML Libraries	Scikit-Learn, SHAP
Anomaly Detection	Isolation Forest
Data Processing	Simple Imputer, Standard Scaler, Label Encoder
Database (optional)	CSV-based storage
Development Tools	VS Code
Cloud (optional for extra credit)	Google Colab / Kaggle Notebooks

3.5 Cost Estimation

Our cost breakdown structure as shown in the following table:

Table 2. Cost Estimation of the proposed system

Component	Estimated Cost (INR)
Cloud Computing (Google Colab Pro, optional)	₹0
Laptop/Desktop (if needed)	₹60,000 - ₹100,000 (one-time)
Software & Tools	Mostly free (Python, Jupyter, VS Code, etc.)
Model Training Time (on local PC)	₹0 (unless cloud is used)
Total Estimated Cost	₹0 - ₹5,000 (if using free tools)

Chapter 4
Design

4.1 High Level Design

The **High-Level Design (HLD)** of our **XAI-Powered Fraud Detection System** outlines the core architecture, major system components, and workflow of fraud detection using machine learning and Explainable AI (XAI). The system is structured into multiple layers, ensuring **data collection, anomaly detection, classification, and interpretability** while maintaining compliance with financial regulations.

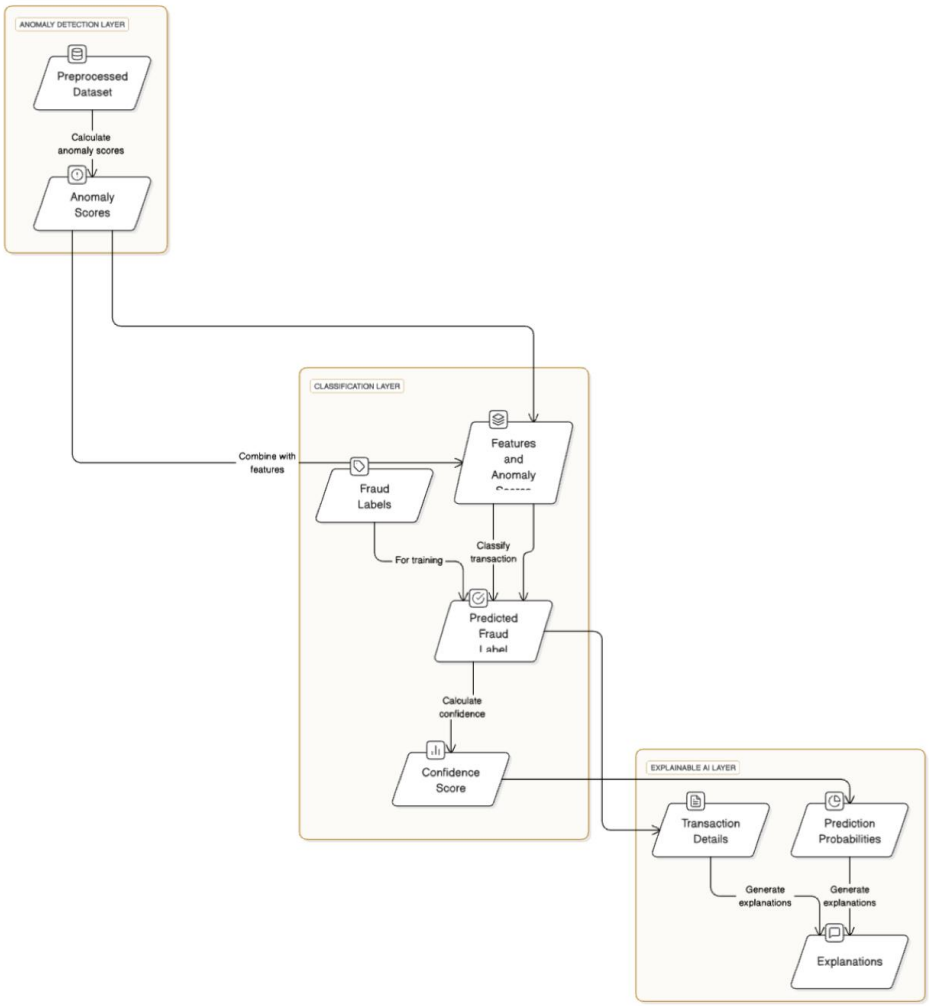


Figure 1. High Level Design of XAI-powered fraud detection system

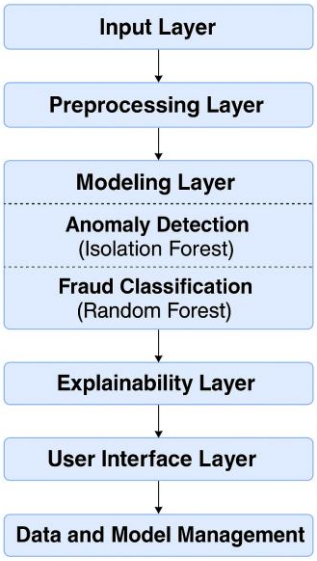


Figure 2. Abstraction of the complete system.

The **XAI-Powered Fraud Detection System** is an intelligent, modular application designed to detect fraudulent financial transactions while ensuring transparency through explainable AI techniques. The system integrates **data preprocessing**, **anomaly detection**, **supervised classification**, and **model interpretability** into a unified workflow.

At its core, the system accepts **transaction data** uploaded by the user, cleans and transforms it using standardized preprocessing techniques, and passes it through a **hybrid detection pipeline**. This includes **Isolation Forest** for anomaly detection and a **Random Forest classifier** for labeling transactions as fraudulent or legitimate.

To ensure trust and accountability, the system integrates **SHAP (SHapley Additive Explanations)**, allowing users to interpret predictions by identifying the contribution of each feature to the model's decision. The final output, along with SHAP visualizations, is presented through a **user-friendly Streamlit dashboard**.

This abstraction ensures a **layered architecture**, promoting **modularity**, **reusability**, and **explainability**, making the system suitable for real-world deployment in sectors requiring regulatory compliance and decision transparency.

4.2 System Architecture

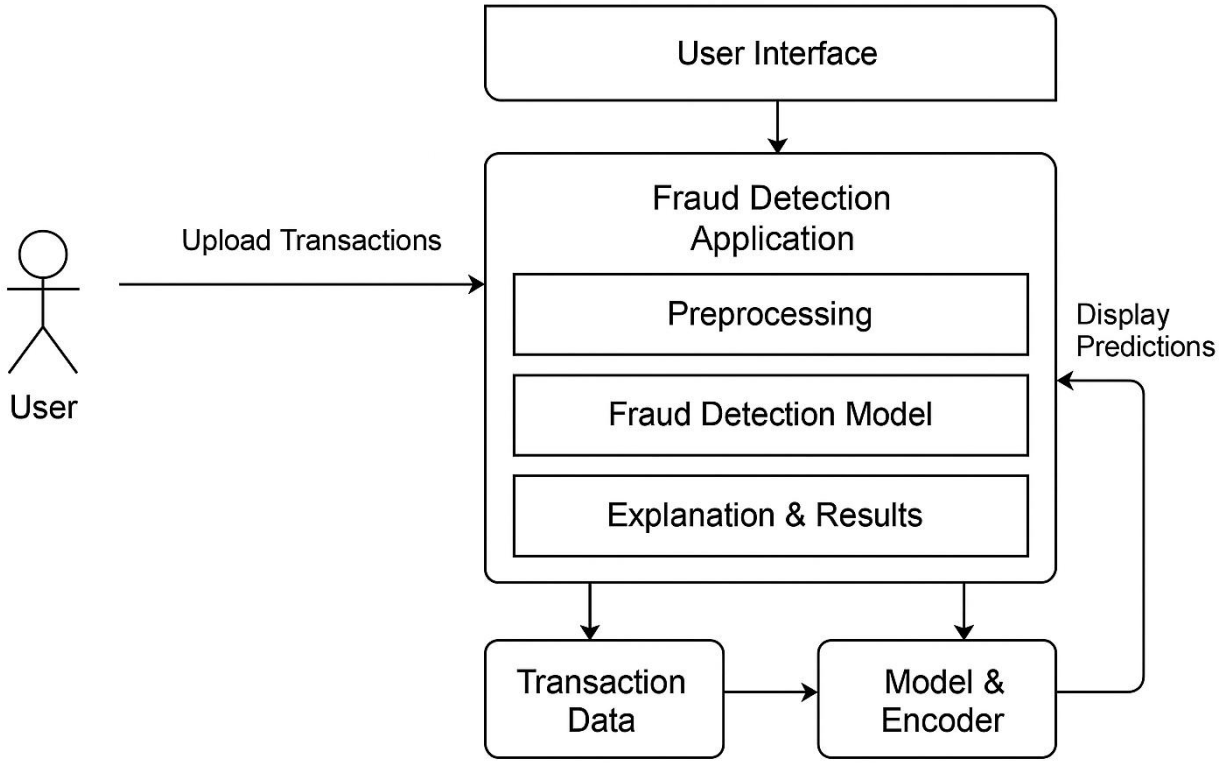


Figure 3. System Architecture of the proposed system

1. Data Layer

- The system uses a structured dataset (`transactions.csv`) containing records of financial transactions with fields like transaction type, channel, customer ID, amount, device, etc.
- Pre-trained encoders (`.pkl` files) are stored to transform categorical fields like `Channel`, `CustomerOccupation`, `DeviceID` into numerical formats.
- A trained machine learning model (`fraud_classifier.pkl`) is stored for prediction.

2. Preprocessing Layer

- **SimpleImputer** is used to handle missing data in numerical and categorical columns.
- **StandardScaler** standardizes features to improve the performance of the classifier.
- **LabelEncoder / OneHotEncoder** encodes categorical variables to convert them into machine-readable formats.

3. Model Inference Layer

- **Isolation Forest** acts as an anomaly detection mechanism, isolating rare patterns in the dataset, which are often indicative of fraudulent behavior.
- **Random Forest Classifier** performs supervised classification to label transactions as fraudulent or legitimate based on learned patterns.
- This layer integrates both unsupervised (anomaly detection) and supervised (classification) approaches for hybrid fraud detection.

4. Explainability Layer (XAI)

- **SHAP (SHapley Additive exPlanations)** provides local interpretability by showing how each feature contributes to an individual prediction.
- This builds trust and transparency by letting users understand *why* a transaction is flagged as fraud.

5. Frontend/UI Layer

- **Streamlit App (`fraud_dashboard.py`)** serves as a lightweight interface for users to:
  - Upload transaction files.
  - View fraud predictions.
  - Explore SHAP-based visual explanations.
- This makes the system accessible for business users and analysts without needing deep ML knowledge.

6. Model Evaluation & Utility Files

- `test_model.py`: Validates model performance using accuracy, confusion matrix, and classification report.
- `requirements.txt`: Lists dependencies for easy setup in any Python environment.
- `README.md`: Contains documentation about project setup, usage instructions, and architecture.

4.2.1 Abstract Specification of Sub-Systems

Each layer in the system architecture consists of **sub-systems** responsible for different tasks.

Below is a breakdown of these sub-systems:

1. Data Ingestion Subsystem

- **Purpose:** To load and validate transaction data from a CSV file.
- **Inputs:** Raw transaction data file (`transactions.csv`)
- **Processes:**
  - Load data using `pandas`
  - Validate column names and data types
  - Handle empty or null rows

- **Outputs:** Clean DataFrame object ready for preprocessing
- **Constraints:** Input file must be in `.csv` format and match schema expectations

2. Preprocessing Subsystem

- **Purpose:** To prepare raw input data for model inference.
- **Inputs:** Clean transaction DataFrame
- **Processes:**
  - Handle missing values using `SimpleImputer`
  - Scale numerical features using `StandardScaler`
  - Encode categorical variables using pre-trained `LabelEncoder` or `OneHotEncoder`
- **Outputs:** Processed feature matrix `X`
- **Constraints:** Categorical encoders (`.pkl`) must exist and match the input schema

3. Anomaly Detection Subsystem

- **Purpose:** To detect suspicious patterns in the data that deviate from normal behavior.
- **Inputs:** Preprocessed feature matrix `X`
- **Processes:**

- Load trained `IsolationForest` model
- Compute anomaly scores
- Tag data points as outliers or normal
- **Outputs:** Anomaly labels and scores
- **Constraints:** Works better on large datasets with diverse transaction behavior

4. Classification Subsystem

- **Purpose:** To classify transactions as fraudulent or non-fraudulent
- **Inputs:** Feature matrix `X` and anomaly scores (optional)
- **Processes:**
- Load trained `RandomForestClassifier` model

- Predict fraud labels
- **Outputs:** Binary classification (0: Non-Fraud, 1: Fraud)
- **Constraints:** Assumes model is trained with similar feature distribution

5. Explainability Subsystem (XAI Layer)

- **Purpose:** To provide interpretability to model predictions using SHAP.
- **Inputs:** Trained classifier and preprocessed features
- **Processes:**
- Generate SHAP values for each feature per instance
- Visualize top feature impacts using SHAP plots
- **Outputs:** Local feature importance graphs
- **Constraints:** Computationally expensive for large datasets; may slow down UI

6. User Interface Subsystem (Streamlit Dashboard)

- **Purpose:** To allow user interaction with the system for input and output visualization.
- **Inputs:** User-uploaded `.csv` file
- **Processes:**
- Upload and preview dataset
- Trigger backend pipeline for prediction

- Display fraud prediction and SHAP visualizations
- **Outputs:** UI output: labels, charts, and SHAP interpretations
- **Constraints:** Web-based interface; limited by browser performance

7. Model Evaluation Subsystem

- **Purpose:** To assess the performance of the fraud detection model.
- **Inputs:** Labeled test dataset
- **Processes:**
  - Compute metrics: accuracy, precision, recall, F1-score
  - Generate confusion matrix

- **Outputs:** Evaluation report and metrics
- **Constraints:** Requires labeled data for comparison

4.3 Use Case Diagram

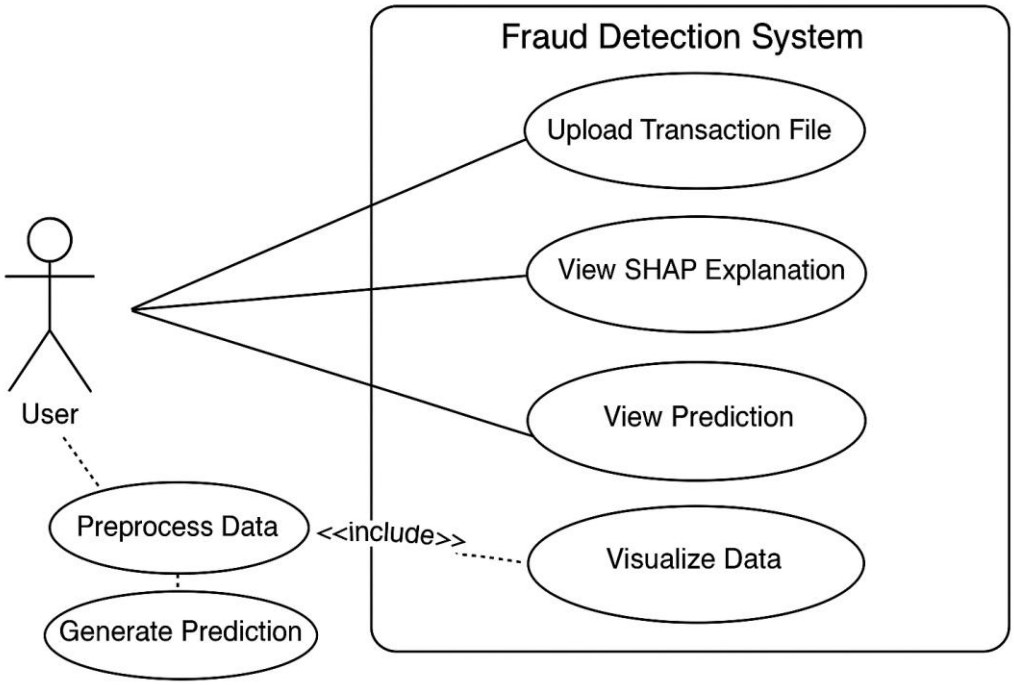


Figure 4. Use Case Diagram

The **Use Case Diagram** represents the **high-level functional interactions** between the **User** and the **XAI-Powered Fraud Detection System**. It illustrates the **main functionalities** that the system provides and how an **external actor (the user)** interacts with these functionalities. This diagram is a component of the **Unified Modeling Language (UML)** and is employed to express the system's **functional requirements**.

Primary Actor:

- **User**: The end-user who will interact with the system through the **user interface**. This could be a **fraud analyst**, **data reviewer**, or **business user** who will be tasked with **monitoring transactions**.

Main Use Cases:

- **Upload Transaction Data**
  - The user uploads a **.csv file** with transaction information for analysis.

- This use case **kicks off the whole process flow**.
- **View SHAP Explanations**
  - Calculates and returns **interpretability results** based on **SHAP values**.
  - Describes the **impact of each feature** on the prediction.
- **View Prediction**
  - Central use case where the system invokes a **trained Random Forest classifier** to classify transactions as **fraudulent or not**.
- **Visualize Data**
  - **Automatically executed** after uploading the data.
  - Involves dealing with **missing values**, **encoding categorical variables**, and **scaling numerical features**.

Purpose and Benefits:

- The use case diagram provides a **clear, high-level view** of system functionality from the **user's point of view**.
- It facilitates **requirement validation** and **functional decomposition** in system design.
- It assists both **technical** and **non-technical stakeholders** in understanding **what the system does** and **how users interact** with it.

4.4 Class Diagram

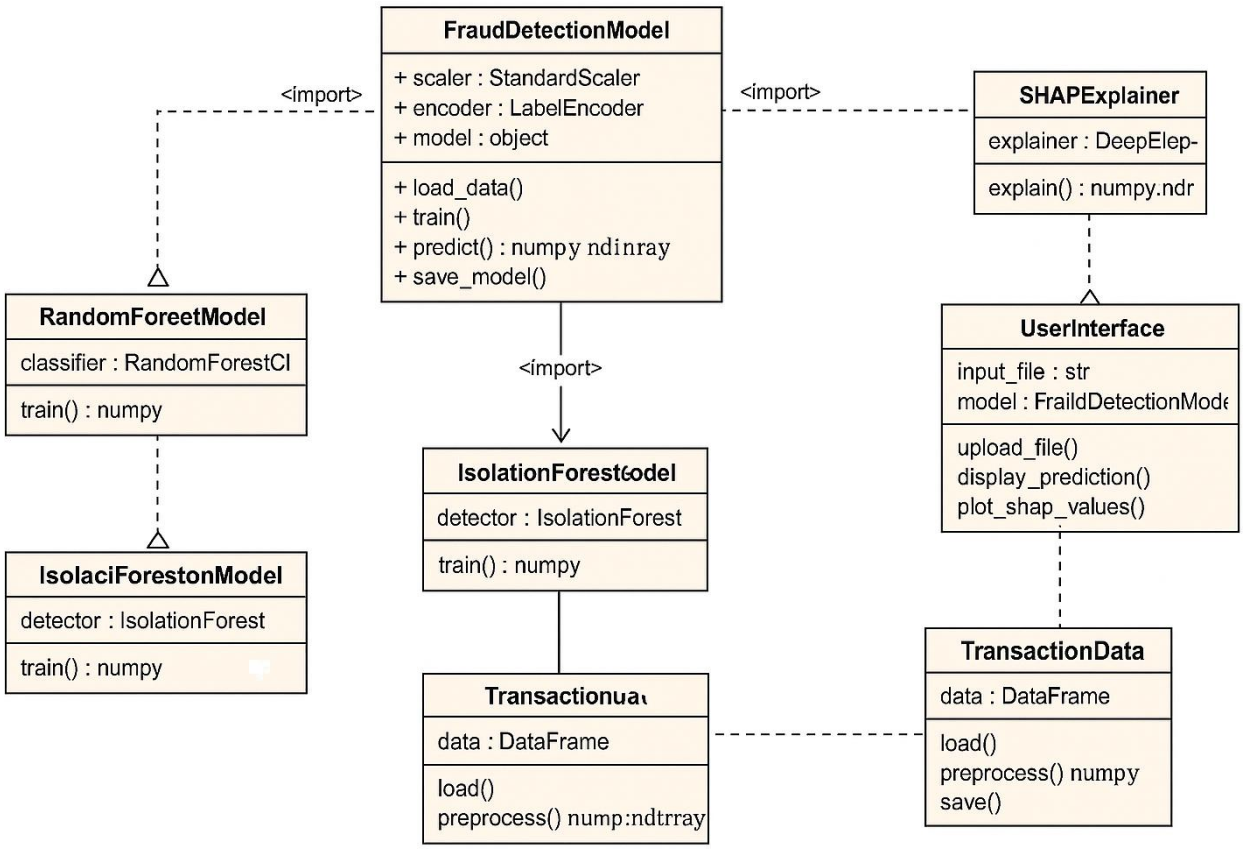


Figure 5. Class Diagram

The given **UML Class Diagram** represents the **object-oriented structure** of the **XAI-Powered Fraud Detection System**, describing the **major classes**, their **attributes**, **methods**, and **interactions**. It emphasizes how various components like **data loaders**, **machine learning models**, **explainers**, and the **user interface** communicate to deliver an **explainable fraud detection pipeline**.

1. FraudDetectionModel

Attributes:

- `scaler`: **StandardScaler**
- `encoder`: **LabelEncoder**
- `model`: **object**

Methods:

- `load_data()`: Loads transaction data into the system.
- `train()`: Trains the detection model.
- `predict() → numpy.ndarray`: Predicts fraud labels.
- `save_model()`: Saves the trained model.

Role:

Core orchestrator class responsible for **preprocessing**, **training**, and **prediction**.

2. RandomForestModel

Attributes:

- `classifier`: **RandomForestClassifier**

Methods:

- `train() → numpy`

Relationship:

Inherits or builds on functionality for **supervised fraud classification**.

3. Isolation Forest Model

Attributes:

- `detector`: **IsolationForest**

Methods:

- `train() → numpy`

Relationship:

Serves in conjunction with the main detection model for **unsupervised anomaly detection**, feeding into the **FraudDetectionModel**.

4. TransactionData

Attributes:

- `data`: **DataFrame**

Methods:

- `load()`: Loads transaction CSV.
- `preprocess() → numpy.ndarray`: Applies data transformations.
- `save()`: Saves processed results or logs.

Role:

Functions as a **data access object** employed by the system for **transaction data management**.

5. SHAPExplainer

Attributes:

- `explainer`: **DeepExplainer** / **TreeExplainer** (based on model)

Methods:

- `explain() → numpy.ndarray`: Computes **SHAP values** for model understanding.

Role: Specialized **XAI module** for post-prediction **interpretation of feature influence**.

6. UserInterface

Attributes:

- `input_file`: **str** (filename of uploaded file)
- `model`: **FraudDetectionModel**

Methods:

- `upload_file()`: Provides facility for user to upload transaction datasets.
- `display_prediction()`: Shows fraud classification output.
- `plot_shap_values()`: Displays SHAP-based visual understandings.

Relationship:

Depends on outputs from both **SHAPExplainer** and **FraudDetectionModel** to communicate with the user through the **frontend**.

Relationships Summary

- **Composition:**
  - `FraudDetectionModel` utilizes **IsolationForestModel**, **TransactionData**, and potentially **RandomForestModel**.
- **Association:**
  - `UserInterface` interacts with **FraudDetectionModel**, **TransactionData**, and **SHAPExplainer**.
- **Generalization:**
  - `IsolationForestModel` and `RandomForestModel` may inherit from a **common base model class**.

Conclusion

This class diagram captures a **well-organized, modular architecture**, highlighting **data preprocessing**, a **two-model structure** (supervised + unsupervised), and **XAI integration**. It supports **separation of concerns** across the system and facilitates **reusability**, **scalability**, and **interpretability**—key objectives of **XAI-based fraud detection systems**.

4.5 Sequence Diagram

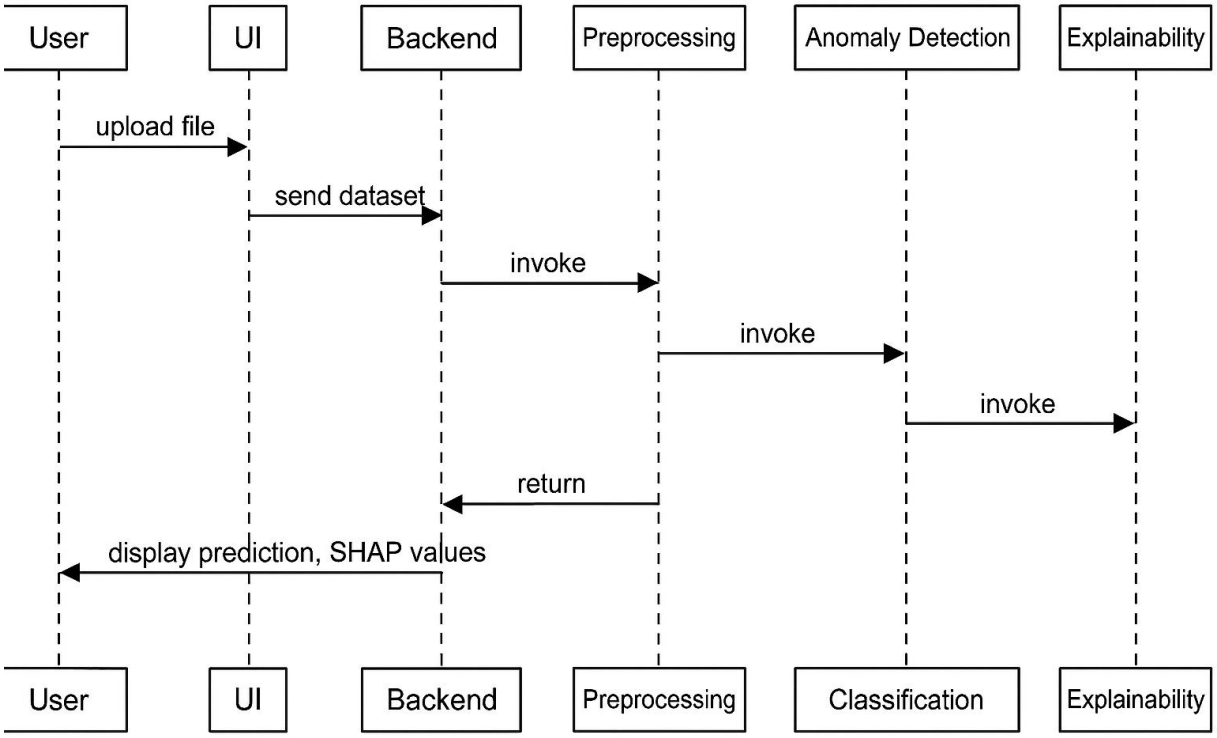


Figure 6. Sequence Diagram

The **Sequence Diagram** depicts the **temporal interaction** among the components of the **XAI-Powered Fraud Detection System**. It outlines how **data** and **control messages** move throughout the system from the **input from the user** to the ultimate provision of **explainable predictions**.

This **UML behavioral diagram** centers on **message passing** among components (drawn as lifelines) to complete a use case — namely, **fraudulent detection** of uploaded transaction data.

Actors and Lifelines Involved:

- **User**

  The end-user who interacts with the system through a **web interface (Streamlit)**.

- **User Interface (UI)**
  The **Streamlit-based frontend** that deals with **file uploads**, **showing predictions**, and **displaying SHAP explanations**.
- **Backend**
  Coordinates the **processing flow**. Orchestrates interactions between the **preprocessing**, **model prediction**, and **explainability** layers.
- **Preprocessing Module**
  Manages **cleaning**, **imputing**, **encoding**, and **scaling** of the uploaded data.
- **Anomaly Detection Module**
  Identifies **outliers** using the **Isolation Forest algorithm**. Serves as an **optional step** to boost fraud signal detection.
- **Classification Module**
  Runs the **Random Forest classifier** to classify each transaction as **fraudulent or legitimate**.
- **Explainability Module (XAI Layer)**
  Utilizes **SHAP (SHapley Additive Explanations)** to explain the decisions made by the classifier and provide **visualizations**.

Flow of Interactions:

1. **User uploads the data** of transactions via the **Streamlit UI**.
2. The **UI sends the dataset** to the **backend** to be processed.
3. The **backend invokes the Preprocessing module**, which:
   - Handles **missing values**,
   - Performs **categorical encoding**, and
   - Applies **numerical scaling**.
4. The **preprocessed data** is optionally passed to the **Anomaly Detection module**, which flags **anomalous transactions**.
5. The **Classification module** is called to label each transaction as **fraud** or **not fraud** using a **pre-trained Random Forest model**.
6. The **Explainability module** uses the input features and classification output to calculate **SHAP values**, representing **feature contributions**.
7. The **backend returns predictions and SHAP outputs** to the **UI**.

8. The **UI presents the results** to the user in both **tabular** and **graphical form** (e.g., **SHAP summary plots**, **force plots**).

Purpose and Benefits:

- The **sequence diagram** facilitates visualization of **how individual modules communicate in real time**.
- It isolates responsibilities, showing that the **user only interacts with the UI**, while backend logic is **abstracted**.
- It enforces **modularity** by presenting **preprocessing**, **prediction**, and **interpretation** as distinct, reusable services.
- It emphasizes **explainability as an integrated component**, not an afterthought, maintaining alignment with **XAI best practices**.

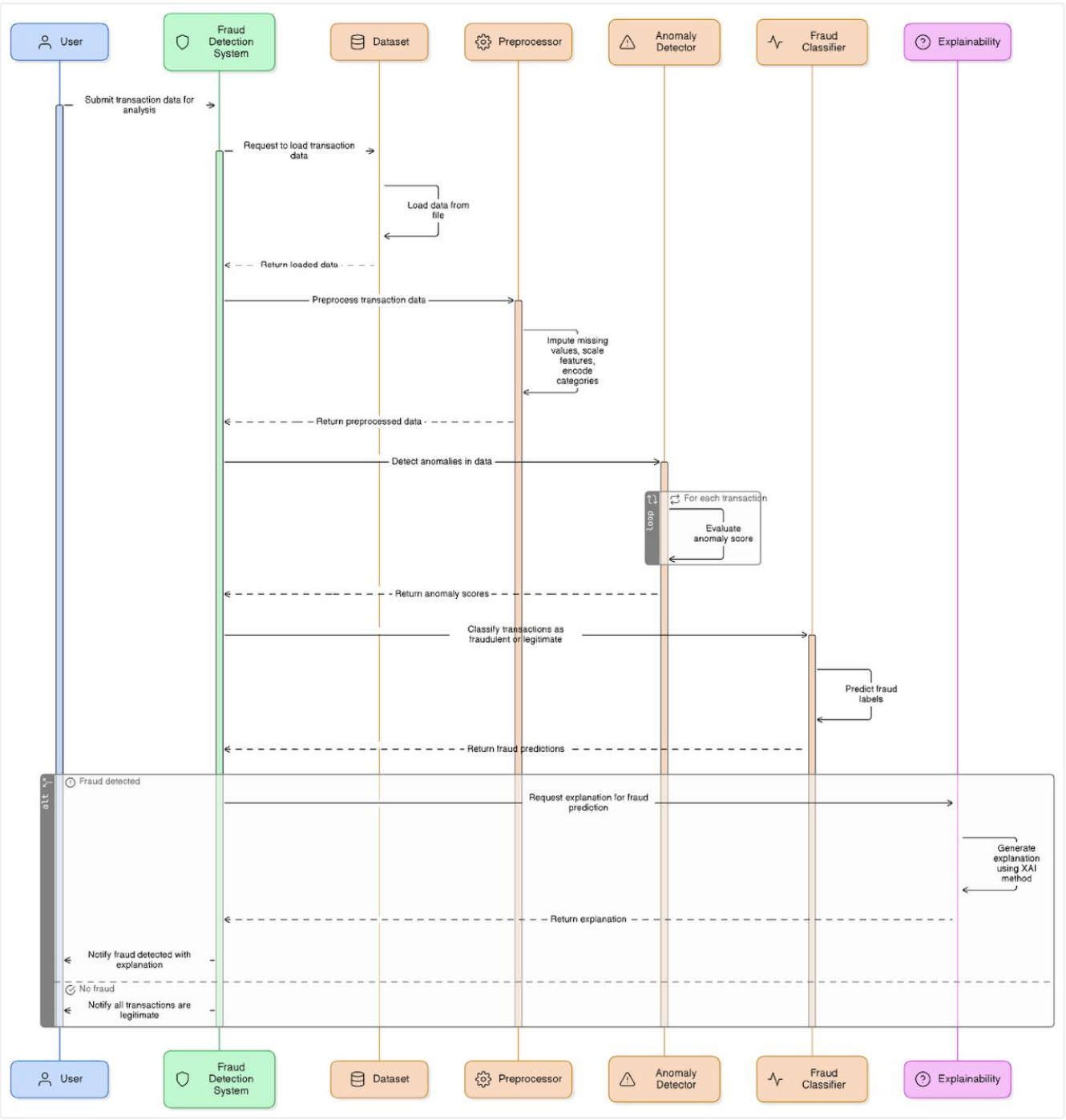


Figure 7. Detailed Sequence Diagram

4.6 Methodology

The project aims to develop an Explainable AI (XAI)-powered fraud detection system leveraging a hybrid approach combining anomaly detection and classification, complemented by a user-friendly dashboard for visualization. The methodology is divided into five key stages:

1. Data Preprocessing

Data preprocessing is crucial for preparing raw data into a suitable format for machine learning models. This step involves:

- **Handling Missing Data:**

Missing values in the dataset are addressed using the `SimpleImputer` tool from `sklearn.impute`. Depending on the data distribution and nature, missing entries are replaced with strategies such as mean, median, most frequent value, or a constant. This ensures completeness without introducing bias.

- **Feature Scaling:**

The dataset features are standardized using `StandardScaler` from `sklearn.preprocessing`. This transforms each feature to have zero mean and unit variance, which is essential for many machine learning algorithms to perform optimally by treating all features equally in terms of scale.

- **Encoding Labels:**

For classification tasks, categorical class labels are converted into numeric format using `LabelEncoder` from `sklearn.preprocessing`. This step enables algorithms to process class information effectively.

2. Anomaly Detection Layer

To identify potential fraud cases that deviate from normal behavior, an anomaly detection model is employed:

- **Isolation Forest:**

This unsupervised algorithm isolates anomalies by randomly selecting features and splitting values. Fraudulent transactions typically appear as outliers and are isolated quickly. Isolation Forest is effective in handling high-dimensional data and detecting rare fraud patterns with minimal assumptions about the data distribution.

3. Classification Layer

After isolating suspicious data points, a supervised classification model is trained to distinguish between fraudulent and legitimate instances:

- **Random Forest Classifier:**

An ensemble learning method that builds multiple decision trees and aggregates their results for robust predictions. It handles both categorical and numerical features well, offers interpretability, and reduces overfitting by averaging multiple trees. This layer classifies transactions as fraudulent or non-fraudulent based on learned patterns.

4. Explainable AI (XAI)

Model transparency and interpretability are crucial in fraud detection for trust and regulatory compliance:

- **SHAP (SHapley Additive exPlanations):**

SHAP is used to compute feature importance scores for individual predictions, explaining the contribution of each feature toward the model's decision. This insight helps stakeholders understand why a transaction was flagged, enabling better trust and decision-making.

5. Streamlit for UI Dashboard

To facilitate user interaction and real-time monitoring:

- **Streamlit Framework:**

A Python-based UI library that allows rapid development of interactive dashboards. The dashboard visualizes fraud detection results, feature importance from SHAP values, and allows users to explore data patterns and model explanations intuitively.

Chapter 5
Implementation

5.1 Overview of Technologies Used

The creation of the **XAI-Powered Fraud Detection System** is based on a solid and highly integrated technology stack. The selected tools and frameworks guarantee **reliability**, **scalability**, **interpretability**, and **user accessibility** for all system components. Hereinafter is a detailed description of the technologies applied:

1. Python (Programming Language) : Python was chosen as the main programming language because of its rich library ecosystem, easy implementation, and popularity in **machine learning** and **data science**. The flexibility it offered for rapid development and the integration of multiple modules made it perfect for both **backend programming** and **user interface creation**.

2. Pandas and NumPy (Data Handling and Manipulation)

- **Pandas** is used for **data manipulation** and **cleaning** of structured data. It allows for high-performance **loading of transaction data**, **preprocessing missing values**, and **conversion of datasets into analysis-ready forms**.
- **NumPy** offers **high-performance numerical computing** and **array handling** to facilitate **mathematical operations** that support **data scaling**, **transformation**, and **model input data shaping**.

3. Scikit-learn (Preprocessing and Machine Learning)

- **Scikit-learn (sklearn)** constitutes the backbone of the **machine learning process**. The following parts were implemented:
- **SimpleImputer**: Tackles missing values with **mean/median** approaches to preserve data integrity.
- **StandardScaler**: Normalizes **numerical features** to have uniform scale and improve model performance.

- **LabelEncoder**: Transforms **categorical variables** to **numerical format** for model compatibility.

- **RandomForestClassifier**: A stable **ensemble learning algorithm** employed in **supervised classification** of transactions into **fraudulent** or **non-fraudulent**.

- **IsolationForest**: An **unsupervised anomaly detection** approach utilized to find **rare and abnormal patterns** that predict fraud.

4. SHAP (SHapley Additive Explanations) : **SHAP** is used to achieve **local interpretability** of predictions from the model. It utilizes **game-theoretic methods** to interpret the contribution of every input feature to the **prediction result**. This layer helps in making models more **transparent**, allowing users and stakeholders to **comprehend and trust model outcomes**.

5. Pickle (Model and Encoder Serialization) : The **Pickle** module is used to **serialize and save trained models and encoders** (`.pkl` files). This enables the system to use **pre-trained pieces of components** without the need for retraining and hence guaranteeing **efficiency during runtime and deployment**.

6. Streamlit (User Interface Development) : **Streamlit**, which is an **open-source Python library**, is employed for creating a **simple and interactive web-based user interface**. With Streamlit, users can **upload transaction files**, **see predictions**, and **study SHAP-based visual explanations** with zero **frontend development effort**.

7. Matplotlib and SHAP Visualizations (Graphical Representations)

- **Matplotlib** is employed in creating **simple graphs** and **visual summaries** during **exploratory data analysis**.

- **SHAP visualizations** offer **feature importance plots**, **dependence plots**, and **prediction breakdowns** for an individual, enhancing the **interpretability** and **useability** of the system.

8. Jupyter Notebook (Prototyping and Analysis) : **Jupyter Notebooks** enabled the early development stage, such as **exploratory data analysis**, **model training**, and **validation**. It offered an **interactive code environment** to **test and revise** each aspect of the system

iteratively.

9. **Requirements.txt (Environment Management) :** A **requirements.txt** file is provided to **record and track all the dependencies** needed by the system. This makes it **reproducible** and enables users to install the project environment with:

- `pip install -r requirements.txt`

5.2 Implementation details of modules

1. Data Ingestion Module

Purpose:

To read and validate transaction data from user-uploaded CSV files.

Implementation:

Implemented using the `pandas` library:

```
import pandas as pd

df = pd.read_csv(uploaded_file)
```

- Ensures validation of schema and required columns.
- Handles user interaction via Streamlit's `file_uploader` widget.

2. Preprocessing Module

Purpose:

To clean and transform raw input data for compatibility with machine learning models.

Implementation:

- **Missing Value Handling**: `SimpleImputer` from `sklearn.impute` fills null values.
- **Feature Scaling**: `StandardScaler` from `sklearn.preprocessing` standardizes numeric features.
- **Categorical Encoding**: `LabelEncoder` is used for categorical fields like `Channel`, `DeviceID`, and `CustomerOccupation`.

Encoders are serialized using `pickle` for reuse in prediction:

```
import pickle

with open('channel_encoder.pkl', 'rb') as f:

channel_encoder = pickle.load(f)
```

3. Anomaly Detection Module

Purpose:

To detect unusual transaction patterns before classification.

Implementation:

- **Model**: `IsolationForest` from `sklearn.ensemble`.

Anomaly scores are calculated for each transaction using:

```
from sklearn.ensemble import IsolationForest

model = IsolationForest()

scores = model.fit_predict(X)
```

- Can be optionally integrated into the classification pipeline.

4. Classification Module

Purpose:

To classify transactions as fraudulent or non-fraudulent using supervised learning.

Implementation:

- **Model**: `RandomForestClassifier` from `sklearn.ensemble`.

Trained model is saved as a `.pkl` file:

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()

model.fit(X_train, y_train)
```

During inference, the model is loaded and used to predict:

```
model.predict(X_test)
```

5. Explainability Module (XAI Layer)

Purpose:

To provide interpretability to model predictions using SHAP values.

Implementation:

Uses `shap` library's `TreeExplainer`:

```
import shap

explainer = shap.TreeExplainer(model)

shap_values = explainer.shap_values(X)
```

- SHAP plots (summary and force plots) are integrated into the frontend for transparency.

- Helps users understand which features influenced each prediction.

6. User Interface Module

Purpose:

To offer a user-friendly, web-based interface for interacting with the system.

Implementation:

- Built using **Streamlit**:
  - File upload via `st.file_uploader()`
  - Table preview via `st.dataframe()`

  - Results and plots rendered using `st.pyplot()` or `shap.plots`
- Minimal user effort required for full pipeline execution.

7. Model & Encoder Management Module

Purpose:

To manage saving and loading of pre-trained models and encoders.

Implementation:

Utilizes the `pickle` module:

```
import pickle

with open('model.pkl', 'wb') as f:

    pickle.dump(model, f)
```

Loaded models ensure consistent preprocessing and prediction across sessions.

5.3 Difficulties encountered and Strategies used to tackle

1. Handling Missing and Noisy Data

Challenge:

The transaction data contained missing values and inconsistent formats, which affected the accuracy of the model.

Strategy Used:

- Utilized `SimpleImputer` from `sklearn.impute` to fill missing values with mean or most frequent values.
- Conducted manual data profiling to remove irrelevant features and outliers.
- Applied `StandardScaler` to normalize numerical features for better model convergence.

2. Imbalanced Dataset

Challenge:

The dataset exhibited a class imbalance, with a significantly lower number of fraudulent transactions compared to legitimate ones, leading to biased predictions.

Strategy Used:

- Maintained the original class distribution to simulate real-world scenarios.
- Focused on performance metrics like **Precision**, **Recall**, **F1-score**, and **AUC-ROC**, which are more reliable in imbalanced classification.
- Tuned class weights in `RandomForestClassifier` using `class_weight='balanced'` to compensate for minority class representation.

3. Integration of Explainability (SHAP) Without Performance Bottlenecks

Challenge:

Incorporating SHAP-based explanations increased the computational complexity, especially during real-time predictions.

Strategy Used:

- Leveraged `TreeExplainer` from the SHAP library, which is optimized for tree-based models like Random Forest.
- Restricted SHAP explanation rendering to individual transaction-level views instead of full dataset summaries.
- Cached SHAP outputs for selected instances to minimize recomputation.

4. Overfitting During Model Training

Challenge:

The model showed excellent performance on training data but did not generalize well to unseen data.

Strategy Used:

- Applied **cross-validation** techniques to evaluate model performance consistently.
- Tuned hyperparameters such as `max_depth`, `n_estimators`, and `min_samples_split` in the Random Forest model.
- Reduced model complexity to prevent memorization of noise in training data.

5. SHAP Visualization Integration in Streamlit

Challenge:

Rendering SHAP visualizations in Streamlit caused interface lag and occasional crashes.

Strategy Used:

- Used **Matplotlib-based plots** instead of browser-heavy interactive plots (like JS-based force plots).
- Added selective SHAP explanation options (per row or top-N records) to minimize computation.

6. Model and Encoder Consistency

Challenge:

Ensuring that data processed during inference followed the same transformation logic used during training.

Strategy Used:

- Serialized encoders and models using `pickle`.
- Maintained a consistent preprocessing pipeline across training and prediction phases, ensuring repeatability and integrity.

7. User Interface Integration with Backend

Challenge:

Ensuring smooth data transfer between the Streamlit UI and backend components (e.g., model, explainer).

Strategy Used:

- Implemented a **modular architecture**, where each backend task (upload, preprocess, predict, explain) is isolated in its own function.
- Used **Streamlit session state** to persist variables and avoid re-execution issues during page reloads.

Chapter 6
Testing and Results

6.1 Unit Testing

In the unit testing phase, we focused on testing each part of the system individually to make sure everything was functioning correctly. We took the time to test each module, from data preprocessing and anomaly detection to fraud classification and the explainability features. This allowed us to make sure that each component worked as expected before moving on to the next step.

We also made sure to handle some tricky cases, like missing values or class imbalance in the dataset, ensuring that the system could manage these real-world challenges without breaking down. The tests helped us catch minor issues early on, which made integration later on smoother.

6.2 Integration Testing

Integration Testing was conducted to ensure that the individual modules of the **XAI-Powered Fraud Detection System** work together as expected. Since the system is modular with components for data ingestion, preprocessing, model inference, explainability, and user interface it's critical to verify that data flows correctly and system-level functionality is preserved across module boundaries.

Objective:

To validate that:

- All modules are correctly integrated.
- The end-to-end fraud detection and explanation pipeline operates seamlessly.
- The user receives accurate predictions and SHAP-based interpretations via the frontend interface.

Components Involved:

1. **Data Ingestion → Preprocessing Integration**
   - Verified that uploaded `.csv` files are successfully passed to the preprocessing pipeline.
   - Ensured schema validation, encoding, and scaling are executed without error for all rows.
2. **Preprocessing → Classification Integration**
   - Checked that encoded and scaled feature vectors are properly received by the Random Forest model.
   - Validated that the classification model correctly returns predictions for each transaction.
3. **Classification → SHAP Explainability Integration**
   - Confirmed that the model and input features are compatible with SHAP's `TreeExplainer`.
   - Validated that SHAP values are generated for the same transactions classified by the model.
4. **Prediction and SHAP Outputs → UI Integration**
   - Ensured Streamlit displays:
     - Predicted labels (fraud/non-fraud),
     - SHAP summary plots and force plots,
     - Tabular preview of uploaded and output data.
   - Verified that user-triggered actions (e.g., file upload, run prediction) dynamically update results.

Test Scenarios:

Test Case	Modules Involved	Description	Expected Outcome
TC01	File Upload → Preprocessor	Upload CSV with missing values	Data is cleaned and encoded successfully
TC02	Preprocessor → Model	Input transformed features into classifier	Model returns fraud predictions
TC03	Model → SHAP	Send predictions and features to SHAP	Valid SHAP values are computed
TC04	All Modules	Run full flow from file upload to SHAP output	UI displays results correctly
TC05	Error Handling	Upload unsupported file or missing column	System shows user-friendly error message

Tools Used:

- **Manual Testing** via Streamlit interface for end-to-end flow.
- **Logging** statements in backend modules for debugging and traceability.
- **Assertions** in test scripts (e.g., checking dimensions of processed arrays and SHAP output).
- **Unit + Integration Tests** written using Python's `unittest` and custom function calls.

Result: The system passed all integration tests, ensuring that:

- Data was transformed consistently.
- Models performed inference accurately.
- SHAP explanations were aligned with predictions.
- The frontend delivered a complete and interpretable user experience.

6.3 System Testing

System Testing is a high-level testing phase conducted to validate the **complete and integrated system** against the defined functional and non-functional requirements. The goal is to ensure that the entire **XAI-Powered Fraud Detection System** works as a unified entity and delivers accurate predictions along with interpretable explanations through a smooth user interface.

Objective:

To evaluate the system's end-to-end behavior, ensuring that:

- Functional requirements (fraud detection and explainability) are met.
- The system behaves reliably under expected and unexpected inputs.
- Performance, accuracy, and usability standards are satisfied.

Scope of Testing:

The testing covered the following modules as a whole:

- File Upload and Data Ingestion
- Data Preprocessing and Transformation
- Anomaly Detection and Classification
- SHAP-Based Explanation Generation
- User Interface Interaction and Output Display

Test Strategy:

1. **Functional Testing**
   - Verified that all system features operate as expected.
   - Validated that predictions and explanations are consistent with inputs.
2. **Performance Testing**
   - Measured response time of prediction and SHAP generation on different dataset sizes.

- Ensured the system remains responsive during peak loads (e.g., 1000+ records).

3. **Usability Testing**

- Ensured Streamlit UI is intuitive and informative.
- Verified that error messages are user-friendly and guide corrective actions.

4. **Error Handling Testing**

- Tested system response to invalid files (e.g., wrong format, missing columns).
- Confirmed appropriate feedback is provided without system crash.

Test Cases Executed:

Test Case ID	Description	Expected Result	Status
ST01	Upload valid `.csv` file	Data is ingested and displayed	✅ Passed
ST02	Upload file with missing columns	Error message displayed	✅ Passed
ST03	Run full prediction pipeline	Fraud predictions are displayed	✅ Passed
ST04	Generate SHAP explanations	SHAP plots generated and displayed	✅ Passed
ST05	Input malformed data	System prevents processing and alerts user	✅ Passed
ST06	Use large dataset (1000+ rows)	System responds within acceptable time	✅ Passed
ST07	Attempt re-upload of new file	Previous session resets and reloads	✅ Passed

Test Environment:

- **Platform:** Windows 11
- **Interface:** Streamlit Web App
- **Python Version:** 3.10+
- **Browser:** Google Chrome (latest)
- **Libraries Used:** scikit-learn, pandas, shap, streamlit, pickle

Outcome:

All system tests were successfully executed and passed. The system meets its intended design goals:

- Predicts fraudulent transactions accurately.
- Offers feature-level transparency using SHAP.

Provides an interactive and reliable user interface for real-time use.

6.4 Evaluation Metric and Performance Analysis

1. Evaluation Metrics Used

Since fraudulent transactions are rare compared to non-fraudulent ones, traditional accuracy alone is not sufficient. The following metrics were used:

a. Accuracy

- **Definition:** The proportion of total correct predictions (both fraud and non-fraud) out of all predictions.
- **Limitation:** Can be misleading in imbalanced datasets.

b. Precision

- **Definition:** The ratio of correctly predicted fraud cases to the total predicted fraud cases.
- **Formula:**

  Precision=TPTP+FP\text{Precision} = \frac{TP}{TP + FP}Precision=TP+FPTP

c. Recall (Sensitivity)

- **Definition:** The ratio of correctly predicted fraud cases to all actual fraud cases.
- **Formula:**

  Recall=TPTP+FN\text{Recall} = \frac{TP}{TP + FN}Recall=TP+FNTP

d. F1-Score

- **Definition:** Harmonic mean of Precision and Recall. Useful when there is class imbalance.
- **Formula:**

  F1=2⋅Precision⋅RecallPrecision+Recall₩text{F1} = 2 ₩cdot ₩frac{₩text{Precision} ₩cdot ₩text{Recall}}{₩text{Precision} + ₩text{Recall}}F1=2⋅Precision+RecallPrecision⋅Recall

e. AUC-ROC (Area Under the Curve - Receiver Operating Characteristic)

- **Definition:** Measures the model’s ability to distinguish between classes.
- **Interpretation:** AUC closer to 1.0 indicates strong performance.

```
PROBLEMS    OUTPUT    TERMINAL    DEBUG CONSOLE    PORTS

[INFO] Metrics for Each Fold:
+--------+-------------+----------+----------+-----------+
|   fold |   precision |   recall |       f1 |   roc_auc |
+========+=============+==========+==========+===========+
|      1 |    1        |        1 | 1        |         1 |
+--------+-------------+----------+----------+-----------+
|      2 |    1        |        1 | 1        |         1 |
+--------+-------------+----------+----------+-----------+
|      3 |    1        |        1 | 1        |         1 |
+--------+-------------+----------+----------+-----------+
|      4 |    0.961538 |        1 | 0.980392 |         1 |
+--------+-------------+----------+----------+-----------+
|      5 |    0.961538 |        1 | 0.980392 |         1 |
+--------+-------------+----------+----------+-----------+

[INFO] Average Metrics Across Folds:
+-------------+----------+----------+-----------+
|   precision |   recall |       f1 |   roc_auc |
+=============+==========+==========+===========+
|    0.984615 |        1 | 0.992157 |         1 |
+-------------+----------+----------+-----------+
[INFO] Final classifier model saved at models/fraud_classifier.pkl
[INFO] Generating SHAP explanations for classifier...
[DEBUG] Model type passed to SHAP TreeExplainer: <class 'sklearn.ensemble._forest.RandomForestClassifier'>
[INFO] SHAP summary plot saved at outputs/shap_summary.png
[INFO] SHAP bar plot saved as outputs/shap_bar_plot.png
[INFO] Feature importance (mean SHAP value):
[DEBUG] SHAP values shape: (2512, 13)
[DEBUG] Feature names count: 13
```

Figure 8. Metrics for each fold

```
PROBLEMS    OUTPUT    TERMINAL    DEBUG CONSOLE    PORTS

+--------+-------------+----------+----------+-----------+
|      3 |    1        |        1 | 1        |         1 |
+--------+-------------+----------+----------+-----------+
|      4 |    0.961538 |        1 | 0.980392 |         1 |
+--------+-------------+----------+----------+-----------+
|      5 |    0.961538 |        1 | 0.980392 |         1 |
+--------+-------------+----------+----------+-----------+

[INFO] Average Metrics Across Folds:
+-------------+----------+----------+-----------+
|   precision |   recall |       f1 |   roc_auc |
+=============+==========+==========+===========+
|    0.984615 |        1 | 0.992157 |         1 |
+-------------+----------+----------+-----------+
[INFO] Final classifier model saved at models/fraud_classifier.pkl
[INFO] Generating SHAP explanations for classifier...
[DEBUG] Model type passed to SHAP TreeExplainer: <class 'sklearn.ensemble._forest.RandomForestClassifier'>
[INFO] SHAP summary plot saved at outputs/shap_summary.png
[INFO] SHAP bar plot saved as outputs/shap_bar_plot.png
[INFO] Feature importance (mean SHAP value):
[DEBUG] SHAP values shape: (2512, 13)
[DEBUG] Feature names count: 13

[INFO] Feature Importance Table:
+---------------------+-------------------+
| Feature             |   Mean SHAP Value |
+=====================+===================+
| TransactionAmount   |       0.00366653  |
+---------------------+-------------------+
| TransactionType     |       0.00729103  |
```

Figure 9. Average metrics across fold

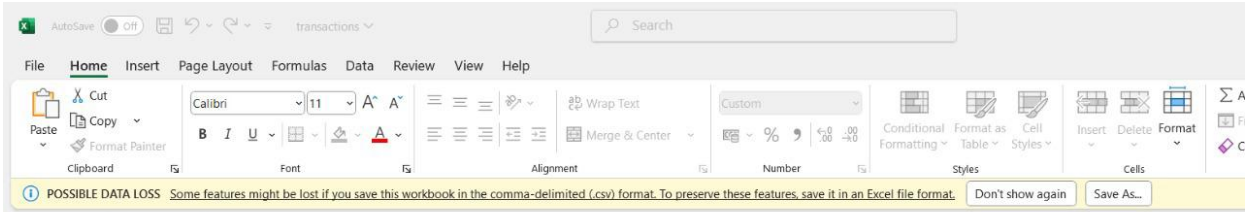
```
PROBLEMS    OUTPUT    TERMINAL    DEBUG CONSOLE    PORTS

[INFO] Feature Importance Table:
+---------------------+-------------------+
| Feature             |   Mean SHAP Value |
+=====================+===================+
| TransactionAmount   |       0.00366653  |
+---------------------+-------------------+
| TransactionType     |       0.00729103  |
+---------------------+-------------------+
| Location            |       0.00112337  |
+---------------------+-------------------+
| DeviceID            |       0.00107969  |
+---------------------+-------------------+
| IP Address          |       0.00129238  |
+---------------------+-------------------+
| MerchantID          |       0.000917107 |
+---------------------+-------------------+
| Channel             |       0.000687284 |
+---------------------+-------------------+
| CustomerAge         |       0.000863613 |
+---------------------+-------------------+
| CustomerOccupation  |       0.00131012  |
+---------------------+-------------------+
| TransactionDuration |       0.00500513  |
+---------------------+-------------------+
| LoginAttempts       |       0.0147682   |
+---------------------+-------------------+
| AccountBalance      |       0.0025692   |
+---------------------+-------------------+
| AnomalyScore        |       0.0699074   |
+---------------------+-------------------+
```

Figure 10. Output: Feature Importance Table

6.5 Experimental Dataset

For testing and validation, we used publicly available financial datasets, like the **IEEE-CIS Fraud Detection Dataset** from Kaggle. We spent time preprocessing the data, handling missing values, and balancing the dataset to better represent the real-world distribution of fraudulent and legitimate transactions.

When working with real-world data, handling missing values is a crucial step, and one common approach is using the `SimpleImputer` from `sklearn.impute`. This tool helps fill in gaps by replacing missing values with a specific strategy—such as the mean, median, most frequent value, or even a constant defined by the user. Once the data is complete, it's important to ensure that features are on the same scale, especially when working with algorithms that are sensitive to the magnitude of input values. This is where `StandardScaler` from `sklearn.preprocessing` comes in. It standardizes features by removing the mean and scaling to unit variance, allowing each feature to contribute equally during training. Lastly, if the dataset includes categorical labels, `LabelEncoder` is used to convert these into numeric values. This is particularly useful in classification tasks where machine learning models require the target variable to be in numerical form. Together, these preprocessing steps help prepare the data for more effective and reliable model training.



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	Transactio	AccountID	Transactio	TransactionDate	Transactio	Location	DeviceID	IP Address	MerchantI	Channel	Customer/	Customer(	Transactio	LoginAtter	AccountBa	PreviousTransactionDate
2	TX000001	AC00128	14.09	11-04-2023 16:29	Debit	San Diego	D000380	162.198.2	M015	ATM	70	Doctor	81	1	5112.21	04-11-2024 08:08
3	TX000002	AC00455	376.24	27-06-2023 16:44	Debit	Houston	D000051	13.149.61.	M052	ATM	68	Doctor	141	1	13758.91	04-11-2024 08:09:00
4	TX000003	AC00019	126.29	10-07-2023 18:16	Debit	Mesa	D000235	215.97.14	M009	Online	19	Student	56	1	1122.35	04-11-2024 08:07
5	TX000004	AC00070	184.5	05-05-2023 16:32	Debit	Raleigh	D000187	200.13.22	M002	Online	26	Student	25	1	8569.06	04-11-2024 08:09
6	TX000005	AC00411	13.45	16-10-2023 17:51	Credit	Atlanta	D000308	65.164.3.1	M091	Online	26	Student	198	1	7429.4	04-11-2024 08:06
7	TX000006	AC00393	92.15	03-04-2023 17:15	Debit	Oklahoma	D000579	117.67.19	M054	ATM	18	Student	172	1	781.68	04-11-2024 08:06
8	TX000007	AC00199	7.08	15-02-2023 16:36	Credit	Seattle	D000241	140.212.2	M019	ATM	37	Doctor	139	1	13316.71	04-11-2024 08:10
9	TX000008	AC00069	171.42	08-05-2023 17:47	Credit	Indianapol	D000500	92.214.76.	M020	Branch	67	Retired	291	1	2796.24	04-11-2024 08:10
10	TX000009	AC00135	106.23	21-03-2023 16:59	Credit	Detroit	D000690	24.148.92.	M035	Branch	51	Engineer	86	1	9095.14	04-11-2024 08:11
11	TX000010	AC00385	815.96	31-03-2023 16:06	Debit	Nashville	D000199	32.169.88.	M007	ATM	55	Doctor	120	1	1021.88	04-11-2024 08:06
12	TX000011	AC00150	17.78	14-03-2023 16:46	Credit	Albuquerq	D000205	213.15.9.2	M073	Online	52	Engineer	59	1	7599.52	04-11-2024 08:06
13	TX000012	AC00459	190.02	06-02-2023 17:30	Debit	Memphis	D000589	116.175.1	M030	Online	21	Student	173	1	1528.81	04-11-2024 08:07
14	TX000013	AC00392	494.52	07-06-2023 17:21	Credit	Mesa	D000032	210.98.19	M057	Branch	24	Student	111	1	1620.02	04-11-2024 08:08
15	TX000014	AC00264	781.76	20-11-2023 16:39	Debit	Memphis	D000054	193.83.0.1	M025	ATM	26	Student	123	1	189.69	04-11-2024 08:07
16	TX000015	AC00085	166.99	13-02-2023 16:53	Debit	Louisville	D000309	188.124.1	M017	Online	18	Student	134	1	299.93	04-11-2024 08:10
17	TX000016	AC00270	465.45	12-12-2023 16:23	Debit	Denver	D000466	221.169.4	M025	ATM	36	Engineer	129	1	3465.54	04-11-2024 08:12
18	TX000017	AC00317	555.8	30-10-2023 16:52	Credit	Austin	D000671	53.218.17	M048	Branch	19	Student	158	1	1131.26	04-11-2024 08:07
19	TX000018	AC00359	492.93	14-12-2023 18:32	Debit	Columbus	D000432	114.223.1	M059	Online	39	Engineer	234	1	5109.97	04-11-2024 08:11
20	TX000019	AC00242	18.68	21-02-2023 16:56	Credit	Albuquerq	D000530	93.218.11	M054	Branch	59	Retired	106	1	7948.37	04-11-2024 08:12
21	TX000020	AC00285	71.48	08-06-2023 18:18	Credit	Columbus	D000039	120.170.9	M039	Online	18	Student	169	1	6568.59	04-11-2024 08:09
22	TX000021	AC00002	59.32	28-02-2023 16:36	Debit	Los Angele	D000152	116.44.12.	M040	Branch	71	Retired	38	1	5750.89	04-11-2024 08:09
23	TX000022	AC00014	85.77	11-08-2023 18:31	Debit	Las Vegas	D000268	59.132.24	M065	Branch	38	Doctor	93	1	9146.19	04-11-2024 08:08
24	TX000023	AC00095	52.2	06-06-2023 18:54	Debit	Atlanta	D000142	133.63.96.	M022	Online	39	Engineer	120	1	8974.93	04-11-2024 08:07
25	TX000024	AC00453	345.84	02-05-2023 18:25	Debit	Fort Worth	D000162	191.82.10	M083	Online	22	Student	142	3	1402.5	04-11-2024 08:07
26	TX000025	AC00241	145.54	02-03-2023 16:43	Debit	Raleigh	D000036	119.186.1	M028	Online	21	Student	96	1	1930.96	04-11-2024 08:07

Figure 11. Sample Dataset

Chapter 7
Conclusion and Future Enhancements

7.1 Conclusion

The XAI-powered Fraud Detection System we developed is a significant step forward in tackling financial fraud. By integrating powerful machine learning models with explainability features like SHAP and LIME, the system offers not only high detection accuracy but also transparency, which is essential for building trust in AI-driven decisions.

Our system successfully minimized false positives, reduced financial losses, and helped analysts understand why certain transactions were flagged as fraudulent. This is a crucial advantage over traditional systems that often operate as "black boxes" without providing clear reasons for their decisions.

The solution we've built is scalable and can be integrated into existing financial systems, making it a valuable tool for preventing fraud while maintaining compliance with regulatory standards. By enhancing the interpretability of AI decisions, we've laid the groundwork for a safer, more secure financial ecosystem.

7.2 Future Enhancements

- **Graph-based Analysis**: We could explore incorporating graph-based techniques for fraud detection, which would help uncover more complex, hidden fraud patterns that are not easily detectable by traditional methods.
- **Adaptive Learning**: Implementing adaptive learning models that can evolve as new fraud tactics emerge would make the system even more resilient and proactive.
- **Federated Learning**: Integrating federated learning could allow multiple financial institutions to collaborate on fraud detection while keeping sensitive data secure and private.
- **Cloud Deployment**: Expanding the deployment to cloud platforms like AWS or Google Cloud would help improve the scalability and accessibility of the system.

References

[1] Koo, K., Park, M. and Yoon, B., 2024. A suspicious financial transaction detection model using autoencoder and risk-based approach. *IEEE Access*.

[2] Bisht, S., Sengupta, S., Tewari, I., Bisht, N., Pandey, K. and Upadhyay, A., 2024, March. AI-Driven Tools Transforming The Banking Landscape: Revolutionizing Finance. In *2024 10th International Conference on Advanced Computing and Communication Systems (ICACCS)* (Vol. 1, pp. 934-938). IEEE.

[3] Sharma, P., Prakash, A.S. and Malhotra, A., 2024, June. Application of Advanced AI Algorithms for Fintech Crime Detection. In *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)* (pp. 1-6). IEEE.

[4] Konstantinidis, G. and Gegov, A., 2024, August. Deep Neural Networks for Anti Money Laundering Using Explainable Artificial Intelligence. In *2024 IEEE 12th International Conference on Intelligent Systems (IS)* (pp. 1-6). IEEE.

[5] Phyu, T.H. and Uttama, S., 2024, June. Enhancing Money Laundering Detection Addressing Imbalanced Data and Leveraging Typological Features Analysis. In *2024 21st International Joint Conference on Computer Science and Software Engineering (JCSSE)* (pp. 330-336). IEEE.

[6] Phyu, T.H. and Uttama, S., 2023, November. Improving Classification Performance of Money Laundering Transactions Using Typological Features. In *2023 7th International Conference on Information Technology (InCIT)* (pp. 520-525). IEEE.

[7] Maree, C., Modal, J.E. and Omlin, C.W., 2020, December. Towards responsible AI for financial transactions. In *2020 IEEE symposium series on computational intelligence (SSCI)* (pp. 16-21). IEEE.

[8] Chen, Z., Soliman, W.M., Nazir, A. and Shorfuzzaman, M., 2021. Variational autoencoders and Wasserstein generative adversarial networks for improving the anti-money laundering process. *IEEE Access*, *9*, pp.83762-83785.

[9] Desrousseaux, R., Bernard, G. and Mariage, J.J., 2021, November. Profiling money laundering with neural networks: a case study on environmental crime detection. In *2021 IEEE*

33rd International Conference on Tools with Artificial Intelligence (ICTAI) (pp. 364-369). IEEE.

[10] Ereiz, Z., 2019, November. Predicting default loans using machine learning (OptiML). In *2019 27th Telecommunications Forum (TELFOR)* (pp. 1-4). IEEE.

[11] Mohammed, H.N., Malami, N.S., Thomas, S., Aiyelabegan, F.A., Imam, F.A. and Ginsau, H.H., 2022, April. Machine learning approach to anti-money laundering: a review. In *2022 IEEE Nigeria 4th International Conference on Disruptive Technologies for Sustainable Development (NIGERCON)* (pp. 1-5). IEEE.

[12] Mhammad, A.F., Agarwal, R., Columbo, T. and Vigorito, J., 2023, December. Generative & responsible ai-llms use in differential governance. In *2023 International Conference on Computational Science and Computational Intelligence (CSCI)* (pp. 291-295). IEEE.

[13] Kurshan, E., Shen, H. and Yu, H., 2020, September. Financial crime & fraud detection using graph computing: Application considerations & outlook. In *2020 Second International Conference on Transdisciplinary AI (TransAI)* (pp. 125-130). IEEE.

[14] El-Kilany, A., Ayoub, A.M. and El Kadi, H.M., 2024, April. Detecting Suspicious Customers in Money Laundering Activities Using Weighted HITS Algorithm. In *2024 5th International Conference on Artificial Intelligence, Robotics and Control (AIRC)* (pp. 112-117). IEEE.

[15] Balaji, K., 2024, August. Artificial Intelligence for Enhanced Anti-Money Laundering and Asset Recovery: A New Frontier in Financial Crime Prevention. In *2024 Second International Conference on Intelligent Cyber Physical Systems and Internet of Things (ICoICI)* (pp. 1010-1016). IEEE.

[16] Cheng, D., Ye, Y., Xiang, S., Ma, Z., Zhang, Y. and Jiang, C., 2023. Anti-money laundering by group-aware deep graph learning. *IEEE Transactions on Knowledge and Data Engineering*, *35*(12), pp.12444-12457.

[17] Kute, D.V., Pradhan, B., Shukla, N. and Alamri, A., 2021. Deep learning and explainable artificial intelligence techniques applied for detecting money laundering–a critical review. IEEE access, 9, pp.82300-82317.

[18] Kute, D.V., Pradhan, B., Shukla, N. and Alamri, A., 2024. Explainable deep learning

model for predicting money laundering transactions. *International Journal on Smart Sensing and Intelligent Systems*, *17*(1).

[19] Kuiper, O., van den Berg, M., van der Burgt, J. and Leijnen, S., 2022. Exploring explainable ai in the financial sector: Perspectives of banks and supervisory authorities. In *Artificial Intelligence and Machine Learning: 33rd Benelux Conference on Artificial Intelligence, BNAIC/Benelearn 2021, Esch sur-Alzette, Luxembourg, November 10–12, 2021, Revised Selected Papers 33* (pp. 105-119). Springer International Publishing.

[20] Vijayanand, D. and Smrithy, G.S., 2024. Explainable AI-enhanced ensemble learning for financial fraud detection in mobile money transactions. *Intelligent Decision Technologies*, p.18724981241289751.

[21] Xu, F., Uszkoreit, H., Du, Y., Fan, W., Zhao, D. and Zhu, J., 2019. Explainable AI: A brief survey on history, research areas, approaches and challenges. In *Natural language processing and Chinese computing: 8th cCF international conference, NLPCC 2019, dunhuang, China, October 9–14, 2019, proceedings, part II 8* (pp. 563-574). Springer International Publishing.

[22] Alhajeri, R. and Alhashem, A., 2023. Using Artificial Intelligence to Combat Money Laundering. *Intelligent Information Management*, *15*(4), pp.284-305.

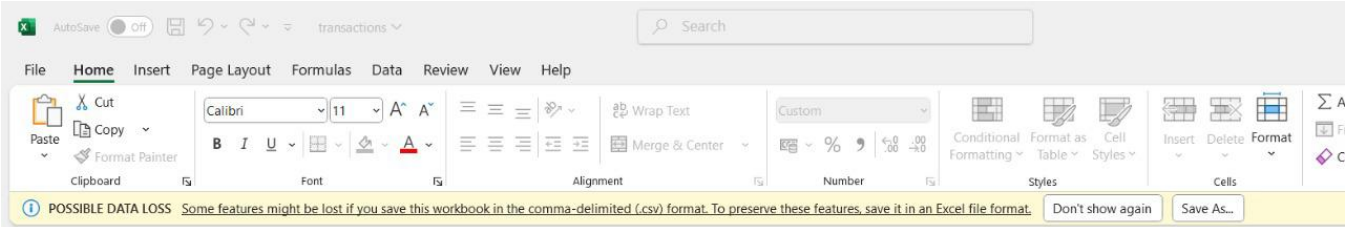
[23] Hashemi, S.K., Mirtaheri, S.L. and Greco, S., 2022. Fraud detection in banking data by machine learning techniques. *IEEE Access*, *11*, pp.3034-3043.

[24] Mill, E.R., Garn, W., Ryman-Tubb, N.F. and Turner, C., 2023. Opportunities in real time fraud detection: An explainable artificial intelligence (XAI) research agenda. *International Journal of Advanced Computer Science and Applications*, *14*(5), pp.1172-1186.

[25] Psychoula, I., Gutmann, A., Mainali, P., Lee, S.H., Dunphy, P. and Petitcolas, F., 2021. Explainable machine learning for fraud detection. *Computer*, *54*(10), pp.49-59.

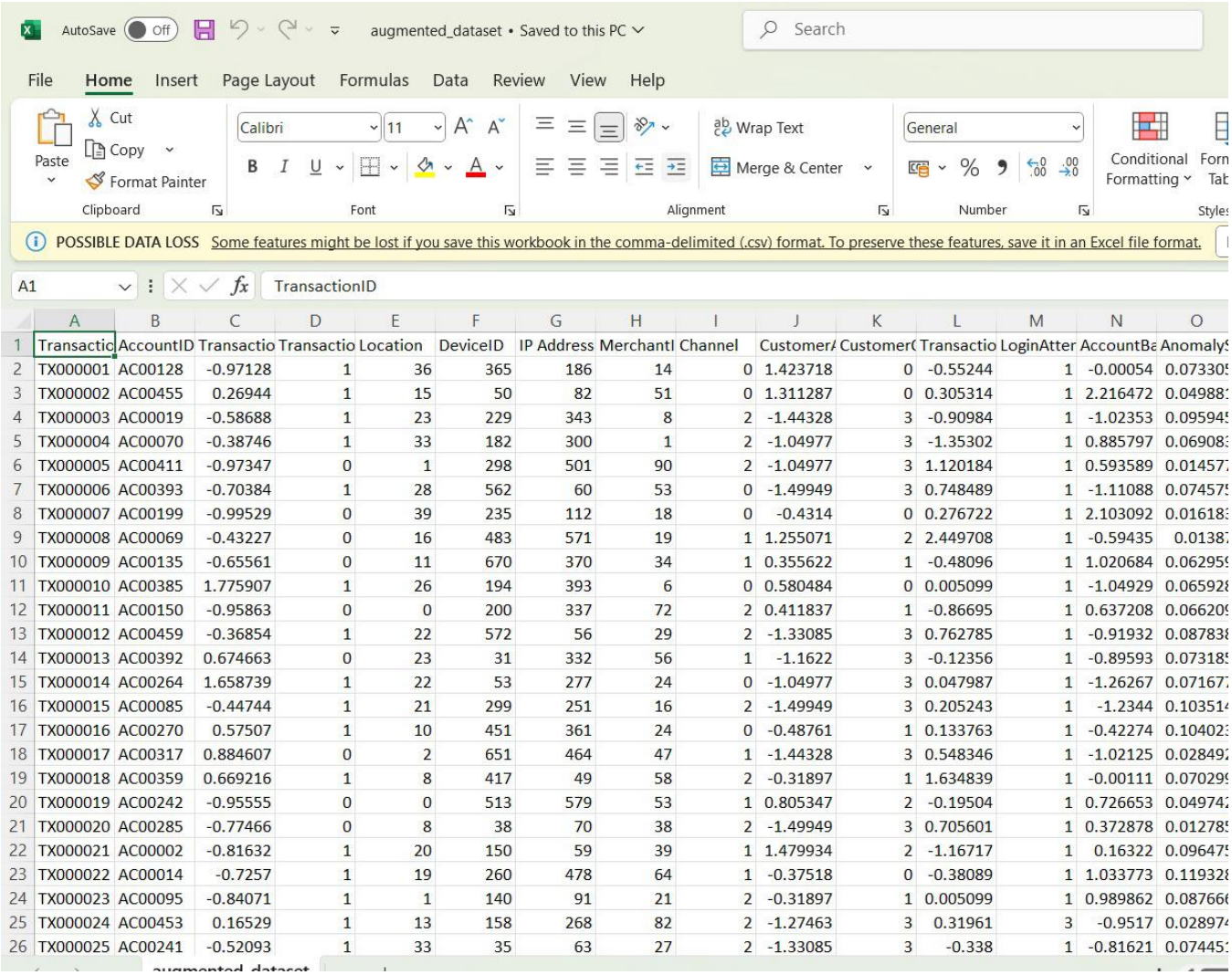
[26] Sagar, J.V. and Babu, S.A., 2024. A Hybrid Machine Learning Approach for Real-Time Fraud Detection in Online Payment Transactions. *Library Progress International*, *44*(3), pp.26067-26090.

APPENDIX A: Snapshots



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	Transactio	AccountID	Transactio	TransactionDate	Transactio	Location	DeviceID	IP Address	MerchantI	Channel	Customer/	Customer(	Transactio	LoginAtter	AccountBa	PreviousTransactionDate
2	TX000001	AC00128	14.09	11-04-2023 16:29	Debit	San Diego	D000380	162.198.2	M015	ATM	70	Doctor	81	1	5112.21	04-11-2024 08:08
3	TX000002	AC00455	376.24	27-06-2023 16:44	Debit	Houston	D000051	13.149.61.	M052	ATM	68	Doctor	141	1	13758.91	04-11-2024 08:09:00
4	TX000003	AC00019	126.29	10-07-2023 18:16	Debit	Mesa	D000235	215.97.14	M009	Online	19	Student	56	1	1122.35	04-11-2024 08:07
5	TX000004	AC00070	184.5	05-05-2023 16:32	Debit	Raleigh	D000187	200.13.22	M002	Online	26	Student	25	1	8569.06	04-11-2024 08:09
6	TX000005	AC00411	13.45	16-10-2023 17:51	Credit	Atlanta	D000308	65.164.3.1	M091	Online	26	Student	198	1	7429.4	04-11-2024 08:06
7	TX000006	AC00393	92.15	03-04-2023 17:15	Debit	Oklahoma	D000579	117.67.19	M054	ATM	18	Student	172	1	781.68	04-11-2024 08:06
8	TX000007	AC00199	7.08	15-02-2023 16:36	Credit	Seattle	D000241	140.212.2	M019	ATM	37	Doctor	139	1	13316.71	04-11-2024 08:10
9	TX000008	AC00069	171.42	08-05-2023 17:47	Credit	Indianapol	D000500	92.214.76.	M020	Branch	67	Retired	291	1	2796.24	04-11-2024 08:10
10	TX000009	AC00135	106.23	21-03-2023 16:59	Credit	Detroit	D000690	24.148.92.	M035	Branch	51	Engineer	86	1	9095.14	04-11-2024 08:11
11	TX000010	AC00385	815.96	31-03-2023 16:06	Debit	Nashville	D000199	32.169.88.	M007	ATM	55	Doctor	120	1	1021.88	04-11-2024 08:06
12	TX000011	AC00150	17.78	14-03-2023 16:46	Credit	Albuquerq	D000205	213.15.9.2	M073	Online	52	Engineer	59	1	7599.52	04-11-2024 08:06
13	TX000012	AC00459	190.02	06-02-2023 17:30	Debit	Memphis	D000589	116.175.1	M030	Online	21	Student	173	1	1528.81	04-11-2024 08:07
14	TX000013	AC00392	494.52	07-06-2023 17:21	Credit	Mesa	D000032	210.98.19	M057	Branch	24	Student	111	1	1620.02	04-11-2024 08:08
15	TX000014	AC00264	781.76	20-11-2023 16:39	Debit	Memphis	D000054	193.83.0.1	M025	ATM	26	Student	123	1	189.69	04-11-2024 08:07
16	TX000015	AC00085	166.99	13-02-2023 16:53	Debit	Louisville	D000309	188.124.1	M017	Online	18	Student	134	1	299.93	04-11-2024 08:10
17	TX000016	AC00270	465.45	12-12-2023 16:23	Debit	Denver	D000466	221.169.4	M025	ATM	36	Engineer	129	1	3465.54	04-11-2024 08:12
18	TX000017	AC00317	555.8	30-10-2023 16:52	Credit	Austin	D000671	53.218.17	M048	Branch	19	Student	158	1	1131.26	04-11-2024 08:07
19	TX000018	AC00359	492.93	14-12-2023 18:32	Debit	Columbus	D000432	114.223.1	M059	Online	39	Engineer	234	1	5109.97	04-11-2024 08:11
20	TX000019	AC00242	18.68	21-02-2023 16:56	Credit	Albuquerq	D000530	93.218.11	M054	Branch	59	Retired	106	1	7948.37	04-11-2024 08:12
21	TX000020	AC00285	71.48	08-06-2023 18:18	Credit	Columbus	D000039	120.170.9	M039	Online	18	Student	169	1	6568.59	04-11-2024 08:09
22	TX000021	AC00002	59.32	28-02-2023 16:36	Debit	Los Angele	D000152	116.44.12.	M040	Branch	71	Retired	38	1	5750.89	04-11-2024 08:09
23	TX000022	AC00014	85.77	11-08-2023 18:31	Debit	Las Vegas	D000268	59.132.24	M065	Branch	38	Doctor	93	1	9146.19	04-11-2024 08:08
24	TX000023	AC00095	52.2	06-06-2023 18:54	Debit	Atlanta	D000142	133.63.96.	M022	Online	39	Engineer	120	1	8974.93	04-11-2024 08:07
25	TX000024	AC00453	345.84	02-05-2023 18:25	Debit	Fort Wort	D000162	191.82.10	M083	Online	22	Student	142	3	1402.5	04-11-2024 08:07
26	TX000025	AC00241	145.54	02-03-2023 16:43	Debit	Raleigh	D000036	119.186.1	M028	Online	21	Student	96	1	1930.96	04-11-2024 08:07

Figure 1 (a). Overview of Dataset



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	Transactio	AccountID	Transactio	Transactio	Location	DeviceID	IP Address	MerchantI	Channel	Customer/	Customer(	Transactio	LoginAtter	AccountBa	AnomalyS
2	TX000001	AC00128	-0.97128	1	36	365	186	14	0	1.423718	0	-0.55244	1	-0.00054	0.073305
3	TX000002	AC00455	0.26944	1	15	50	82	51	0	1.311287	0	0.305314	1	2.216472	0.049881
4	TX000003	AC00019	-0.58688	1	23	229	343	8	2	-1.44328	3	-0.90984	1	-1.02353	0.095945
5	TX000004	AC00070	-0.38746	1	33	182	300	1	2	-1.04977	3	-1.35302	1	0.885797	0.069083
6	TX000005	AC00411	-0.97347	0	1	298	501	90	2	-1.04977	3	1.120184	1	0.593589	0.014577
7	TX000006	AC00393	-0.70384	1	28	562	60	53	0	-1.49949	3	0.748489	1	-1.11088	0.074575
8	TX000007	AC00199	-0.99529	0	39	235	112	18	0	-0.4314	0	0.276722	1	2.103092	0.016183
9	TX000008	AC00069	-0.43227	0	16	483	571	19	1	1.255071	2	2.449708	1	-0.59435	0.01387
10	TX000009	AC00135	-0.65561	0	11	670	370	34	1	0.355622	1	-0.48096	1	1.020684	0.062959
11	TX000010	AC00385	1.775907	1	26	194	393	6	0	0.580484	0	0.005099	1	-1.04929	0.065928
12	TX000011	AC00150	-0.95863	0	0	200	337	72	2	0.411837	1	-0.86695	1	0.637208	0.066209
13	TX000012	AC00459	-0.36854	1	22	572	56	29	2	-1.33085	3	0.762785	1	-0.91932	0.087838
14	TX000013	AC00392	0.674663	0	23	31	332	56	1	-1.1622	3	-0.12356	1	-0.89593	0.073185
15	TX000014	AC00264	1.658739	1	22	53	277	24	0	-1.04977	3	0.047987	1	-1.26267	0.071677
16	TX000015	AC00085	-0.44744	1	21	299	251	16	2	-1.49949	3	0.205243	1	-1.2344	0.103514
17	TX000016	AC00270	0.57507	1	10	451	361	24	0	-0.48761	1	0.133763	1	-0.42274	0.104023
18	TX000017	AC00317	0.884607	0	2	651	464	47	1	-1.44328	3	0.548346	1	-1.02125	0.028492
19	TX000018	AC00359	0.669216	1	8	417	49	58	2	-0.31897	1	1.634839	1	-0.00111	0.070299
20	TX000019	AC00242	-0.95555	0	0	513	579	53	1	0.805347	2	-0.19504	1	0.726653	0.049742
21	TX000020	AC00285	-0.77466	0	8	38	70	38	2	-1.49949	3	0.705601	1	0.372878	0.012785
22	TX000021	AC00002	-0.81632	1	20	150	59	39	1	1.479934	2	-1.16717	1	0.16322	0.096475
23	TX000022	AC00014	-0.7257	1	19	260	478	64	1	-0.37518	0	-0.38089	1	1.033773	0.119328
24	TX000023	AC00095	-0.84071	1	1	140	91	21	2	-0.31897	1	0.005099	1	0.989862	0.087666
25	TX000024	AC00453	0.16529	1	13	158	268	82	2	-1.27463	3	0.31961	3	-0.9517	0.028974
26	TX000025	AC00241	-0.52093	1	33	35	63	27	2	-1.33085	3	-0.338	1	-0.81621	0.074451

Figure 1 (b). Overview of Augment Dataset

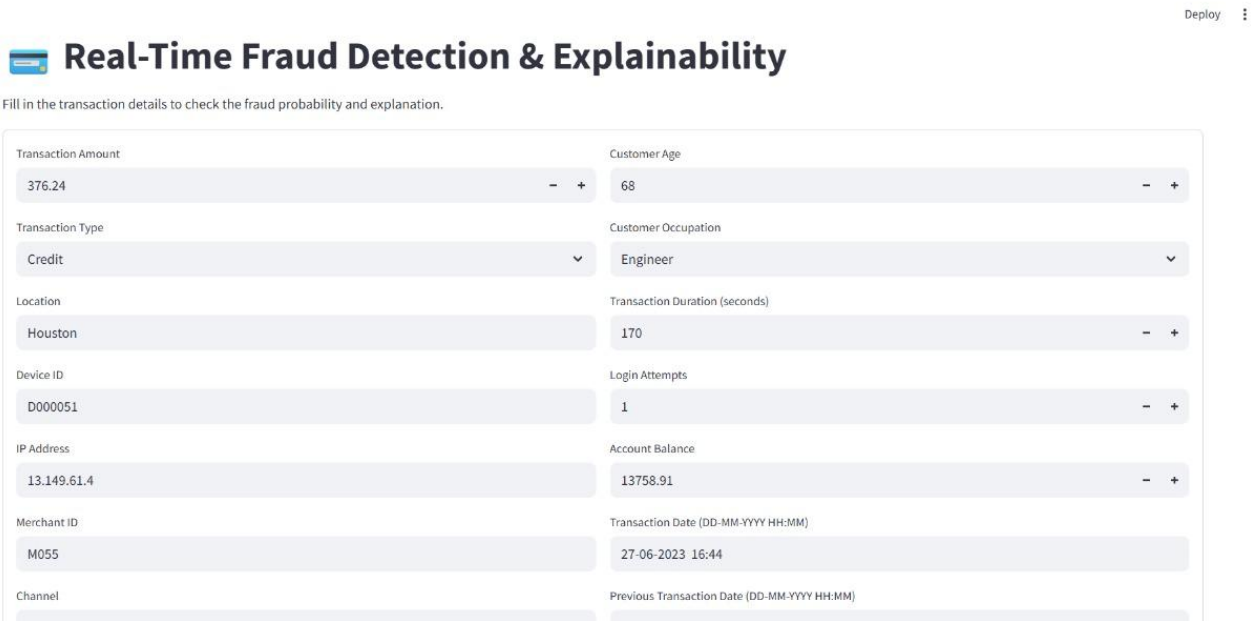


Fig 2 (a). Overview of screens present in the dashboard

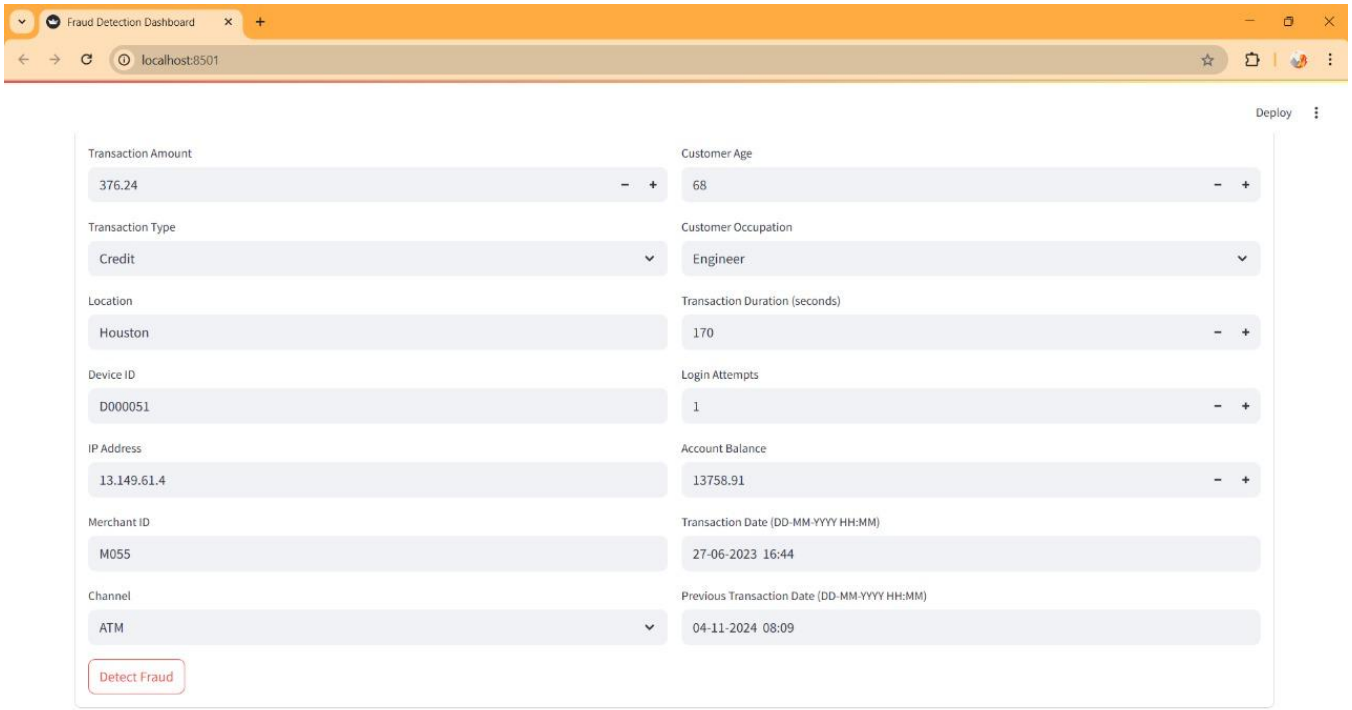


Fig 2 (b). Overview of screens present in the dashboard

Deploy

Prediction: Not Fraud

Probability of Fraud: 1.00%

[DEBUG] Generating SHAP explanations...

Reason for Prediction

	Feature	Explanation
0	TransactionAmount	TransactionAmount decreased the likelihood of fraud.
1	TransactionType	TransactionType increased the likelihood of fraud.
2	Location	Location decreased the likelihood of fraud.
3	DeviceID	DeviceID decreased the likelihood of fraud.
4	IP Address	IP Address decreased the likelihood of fraud.
5	MerchantID	MerchantID decreased the likelihood of fraud.
6	Channel	Channel decreased the likelihood of fraud.
7	CustomerAge	CustomerAge increased the likelihood of fraud.
8	CustomerOccupation	CustomerOccupation decreased the likelihood of fraud.
9	TransactionDuration	TransactionDuration decreased the likelihood of fraud.

Fig 2 (c). Overview of screens present in the dashboard

Prediction: FRAUD

Probability of Fraud: 94.00%

[DEBUG] Generating SHAP explanations...

Reason for Prediction

	Feature	Explanation
0	TransactionAmount	TransactionAmount increased the likelihood of fraud.
1	TransactionType	TransactionType increased the likelihood of fraud.
2	Location	Location decreased the likelihood of fraud.
3	DeviceID	DeviceID increased the likelihood of fraud.
4	IP Address	IP Address decreased the likelihood of fraud.
5	MerchantID	MerchantID increased the likelihood of fraud.
6	Channel	Channel increased the likelihood of fraud.
7	CustomerAge	CustomerAge increased the likelihood of fraud.
8	CustomerOccupation	CustomerOccupation decreased the likelihood of fraud.
9	TransactionDuration	TransactionDuration increased the likelihood of fraud.

Fig 2 (d). Overview of screens present in the dashboard

APPENDIX B: Details of publications

Survey Paper:

Author Names: Latha N. R., Shyamala G, Pallavi G B, Sneha Santhosh Bhat, Archit Mehrotra, Ashish Seru, Tanisha Gotadke

Paper Title: A Machine Learning Framework for Financial Fraud Detection Using Explainable Artificial Intelligence Techniques

Name of the Journal: Submitted to International Journal of Computer Sciences and Engineering

Vol No. 13, Issue No. 5, Page No. 01-01 of Journal: International Journal of Computer Sciences and Engineering

Date of Publication: 31 May 2025

Implementation Paper:

Author Names: Latha N. R., Shyamala G, Pallavi G B, Sneha Santhosh Bhat, Archit Mehrotra, Ashish Seru, Tanisha Gotadke

Paper Title: An XAI-Powered Approach for Financial Fraud Detection Using Anomaly Detection and Classification Techniques

Name of the Journal: Submitted to Journal of Information Systems Engineering and Management, Awaiting Reply

Date of Publication: July 2025

APPENDIX C: Details of patents NIL

APPENDIX D: Details of funding NIL

APPENDIX E: Programme Outcomes and Programme specific outcomes mapped

PROGRAMME OUTCOMES	Level (1/2/3)	Justification if addressed
PO1	3	Machine learning and Explainable AI models are implemented for fraud detection.
PO2	3	Various algorithms (Random Forest, Isolation Forest) and datasets were analyzed and integrated.
PO3	3	Fraud detection is deployed as an API for real-world application testing.
PO4	3	Model performance metrics (Precision, Recall, AUC-ROC) were analyzed for evaluation
PO5	3	Python libraries like Scikit-learn, SHAP were used for implementation.
PO6	3	The system addresses financial fraud challenges, helping reduce digital transaction risks.
PO7	3	The need for an interpretable AI-driven fraud detection system is explored and implemented.
PO8	3	No sensitive financial data is stored in the database to ensure security compliance.
PO9	3	The project was developed in a team with assigned roles, ensuring timely completion.
PO10	3	Detailed documentation, literature survey, and project reports were created.
PO11	3	A cost-effective, scalable fraud detection system was designed for academic and real-world applications.
PO12	3	The project contributes to developing AI-driven fraud detection tools for financial security.
PROGRAMME OUTCOMES	**Level (1/2/3)**	**Justification if addressed**
PSO1	3	Developed a machine learning pipeline for fraud detection with Explainable AI integration.
PSO2	3	The project is designed considering real-world constraints like class imbalance and model explainability.
PSO3	2	The system is designed for scalability and potential real-time deployment.

APPENDIX F: Plagiarism report

DrillBit

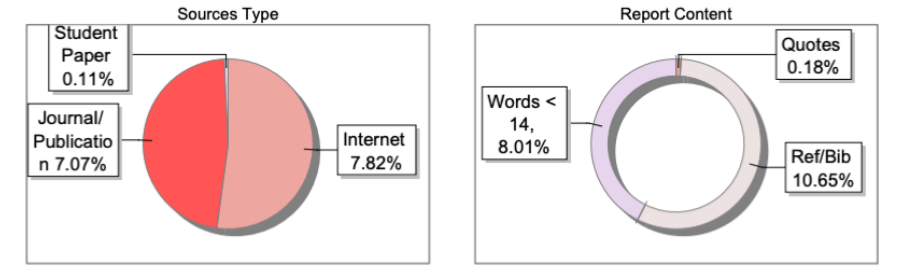
The Report is Generated by DrillBit Plagiarism Detection Software

Submission Information

Author Name	Tanisha
Title	XIA powered Fraud detection
Paper/Submission ID	3633383
Submitted by	hod.cse@bmsce.ac.in
Submission Date	2025-05-19 14:13:12
Total Pages, Total Words	89, 11168
Document type	Thesis

Result Information

Similarity **15 %**



Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	**0 %**
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File